

Secure Computation for Fixed-point and Floating-point Arithmetic

Tianpei Lu

Bingsheng Zhang

Yuyang Feng

Kui Ren

The State Key Laboratory of Blockchain and Data Security, Zhejiang University
Hangzhou, China

lutianpei@zju.edu.cn, bingsheng@zju.edu.cn, 22421104@zju.edu.cn

Abstract

Secure Multi-Party Computation (MPC) protocols naturally operate over rings/fields, and they are less efficient for real-number arithmetics, which are commonly needed in AI-powered applications. State-of-the-art solutions are hindered by the high cost of fixed-point and floating-point operations. This work addresses these bottlenecks by proposing a series of novel MPC protocols. Compared to SOTA, our fixed-point multiplication protocol reduces the online communication cost by about 75%. For scenarios where higher precision is required, we present the first constant-round floating-point arithmetic protocol for addition and multiplication in the three-party computation (3PC) setting, reducing the communication overhead of SOTA by approximately 95%. The experimental results demonstrate that our fixed-point multiplication protocol is more than $3\times$ faster than all mainstream solutions (such as ABY3, Falcon, Orca, etc.). Our floating-point addition and multiplication protocols are over $3\times$ and $5\times$, respectively, faster than SOTA, SecFloat [S&P 23].

1 Introduction

In recent years, secure multi-party computation (MPC) [50, 17, 6] has emerged as an important privacy-enhancing technology that enables secure computation over sensitive data with privacy assurance. MPC protocols naturally operate over finite rings/fields, which makes it hard to efficiently deal with real-number arithmetics – an increasingly common requirement in privacy-sensitive applications such as machine learning [33, 10, 46], data mining [41, 34, 35, 20], and statistical analysis [23, 30, 48], etc. These challenges are primarily caused by the fact that we need to emulate real-number arithmetics using integer operations in the MPC setting, and it is a long-standing research problem on how to find efficient representation and MPC-friendly method to process of fractional values within such discrete structures.

For scenarios where precision is not a critical requirement, fixed-point representations are often used, particularly for privacy-preserving machine learning (PPML), e.g., [36, 34, 11, 13, 53, 45]. This approach is widely adopted by most PPML platforms for linear operations, such as matrix multiplication and convolution. A key challenge in performing fixed-point multiplication within the MPC frameworks arises from the inherent mismatch between the fixed-point representation and the underlying finite ring/field structure. For instance, consider two fixed-point numbers, x and y , each represented with ℓ_i bits for the integer part and ℓ_f bits for the fractional part. To perform multiplication, these numbers are encoded onto the ring, transforming them into $\hat{x} = x \cdot 2^{\ell_f}$ and $\hat{y} = y \cdot 2^{\ell_f}$. The product of these two scaled integers results in a value scaled by $2^{2\ell_f}$, which requires $\ell_i + 2\ell_f$ bits to represent the full-range of the integer result without overflow. As a consequence, the multiplication must be carried out in a ring of at least size $2^{\ell_i+2\ell_f}$ bits to accommodate this expanded range, causing increased communication and computational overhead compared to operations over smaller rings. Furthermore, since the computed result is scaled by $2^{2\ell_f}$, a truncation protocol is needed to restore the original scaling of 2^{ℓ_f} . Conventional solutions typically employ probabilistic truncation protocols [36, 34, 25, 43, 9, 14, 31], but these introduce errors

in the higher-order bits. The occurrence probability of such errors is related to the number of leading zeros, denoted as $\bar{\ell}$, in the higher-order bits of the value being truncated, with an error probability of approximately $2^{-\bar{\ell}-1}$. (See Mohassel *et al.* [36] and Zhou *et al.* [53] for more discussion.) To mitigate this drawback, a larger ring size is necessary to ensure that the multiplication result contains as many leading zeros as possible in the higher bits. In practical scenarios, $\bar{\ell}$ is generally set to at least 32. Putting things together, for fixed-point numbers with ℓ_i bits for the integer part and ℓ_f bits for the fractional part, the multiplication requires a ring size of $\ell_i + 2\ell_f + \bar{\ell}$ bits. As a result, even 16-bit fixed-point numbers often require MPC protocols over a ring of 64 bits or larger.

For scenarios that require higher-precision, we need to realize floating-point arithmetics in the MPC setting, and additional steps, such as alignment, must be performed on top of fixed-point computations. These steps involve complicated non-linear operations, including comparisons and shifts, resulting in a total cost that far exceeds that of fixed-point arithmetics. Existing approaches for floating-point MPC can be broadly classified into two categories. The first category [15, 37] translates floating-point operations into generic Boolean or arithmetic circuits, which are then evaluated using general-purpose MPC protocols. The second category [4, 3, 38, 42, 39] designs custom protocols tailor-made to floating-point representations. While implemented in two-party settings, both approaches heavily rely on cryptographic primitives such as homomorphic encryption [8] and oblivious transfer, incurring substantial communication overhead and round complexity. In terms of round complexity, except garbled circuits [37, 19], none of the existing solutions achieve constant-round complexity, making them impractical in high-latency network environments. Nevertheless, even for garbled circuits, the introduction of the multiplicative factor of the security parameter λ (typically set to 128 or higher) results in huge communication overhead. To the best of our knowledge, no existing work in the literature can achieve constant-round complexity, while making the communication volume independent of the security parameter λ . It severely restricts the practical applicability of floating-point MPC in real-world scenarios.

Our results. In this work, we systematically study the aforementioned problems. We present a series of MPC protocols for fixed-point multiplication, floating-point addition, and floating-point multiplication. Fixed-point multiplication. For fixed-point multiplication, we overcome the issue of needing a $\ell_i + 2\ell_f + \bar{\ell}$ -bit ring for the above-mentioned (ℓ_i, ℓ_f) by reducing it to just $\ell_i + \ell_f + 1$ bits. We observe that 3PC replicated secret sharing [5] has favorable properties. When executing the multiplication protocol in replicated secret sharing, the product is first computed locally in the form of additive secret sharing without any communication, followed by resharing to obtain a replicated secret sharing of the product. Our starting point is that, for fixed-point numbers, we directly use a simplified representation with replicated secret sharing on an $\ell_i + \ell_f$ -bit ring. Then, during the multiplication, we locally extend (non-interactively) it to a sufficiently large ring. By leveraging the properties of replicated secret sharing, we can non-interactively compute the additive secret sharing of the product on this larger ring. We then perform a non-interactively cut on this additive secret sharing, retaining only $\ell_i + \ell_f$ bits, and resharing the $\ell_i + \ell_f$ -bit additive secret sharing to obtain the replicated secret share output. We find that this scheme reduces the communication from $O(\ell_i + 2\ell_f + \bar{\ell})$ to $O(\ell_i + \ell_f)$. The challenge here is that local extension (non-interactively) to a sufficiently large ring can introduce errors due to wrap-around. To address this, we introduced an error elimination mechanism, which adds almost no additional communication overhead in the online phase (cf. Sec. 3). To optimize the communication in the online phase, we employ a variant of replicated secret sharing [12], which shifts part of the online phase cost to the offline phase. Ultimately, our protocol reduces the online phase cost of traditional multiplication and truncation-based protocols (with $2(\ell_i + 2\ell_f + \bar{\ell})$ bits) to $2((\ell_i + \ell_f + 1))$ bits. For a typical 16-bit fixed-point configuration (e.g., $\ell_i = 4$, $\ell_f = 12$, $\bar{\ell} = 32$), our approach reduces online communication by 75%.

Floating-point arithmetics. For floating-point computation, alignment operations need to be performed on top of fixed-point arithmetic. We have implemented constant-round floating-point arithmetic for the first time under secret sharing. Compared to potential constant-round candidates like function-based secret sharing and garbled circuits, whose protocols rely on one-way functions and require security parameters based on hardness assumptions (e.g., $\lambda = 128$), our scheme is the first to achieve information-theoretically secure floating-point computation in constant rounds (though we used a PRG to generate

secret shares to optimize communication, the PRG can be removed by directly sending the secret shares). Thus, achieving much lower communication cost compared to schemes like garbled circuits. The core of the alignment operation lies in finding the first non-zero bit. The key observation in our protocol is as follows: to compute the index (ζ) of the first non-zero bit of $x = m_x + r_x$, we can determine it using the wrap-around bit b of $m_x + r_x = b \cdot 2^\ell + x$. For $b = 0$, since all bits of x before position ζ are 0, it indicates that each corresponding bit of m_x and r_x should be $0 + 0$ to result in 0 in x , rather than $1 + 1$, which would cause a carry and set $b = 1$. From this, we conclude that for $b = 0$, the first position ζ' where $m_{x,(\zeta')} \vee r_{x,(\zeta')} = 1$ ($m_{x,(\zeta')}$ corresponds to ζ' -th bit of m_x) indicates that ζ is either $\zeta' - 1$ or ζ' , depending on whether a carry was generated at bit ζ' . A similar analysis can be done for $b = 1$. Using this observation, we transform the alignment operation into a wrap-around bit calculation, which can then be implemented in a constant-round [32, 44, 26]. Additionally, we have implemented several efficient constant-round protocols, including left-shift protocols and first non-zero bit detection protocols (for a binary list), among others. Ultimately, by combining these protocols, we achieve an efficient constant-round alignment operation. It is worth mentioning that these protocols can also be effectively used in other secure computation scenarios. While Secfloat [40] represents the state-of-the-art in two-party settings, our work introduces significant improvements in communication efficiency for the three-party case. Specifically, our multiplication protocol reduces the communication volume by over 95%, and our floating-point addition protocol reduces it by 87% (see Table 2), with no other open-source solutions available for comparison in the three-party setting.

Performance. Table 1 shows the performance comparison between our fixed-point multiplication and the currently used multiplication-truncation scheme. When the data size is large enough and under typical network conditions, our online phase performance is improved by $2.7 - 3.7\times$. Table 3 and Table 4 demonstrate the performance advantages of our floating-point scheme. Our scheme performs $5\times$ faster in floating-point multiplication and $3\times$ faster in floating-point addition compared to the SOTA.

Paper Organization. Sec. 2 describes some basic preliminaries, while Sec. 3 presents our fixed-point multiplication protocol. Sec. 4 introduces our constant-round secure alignment protocol, which consists of three subprotocols: the first non-zero element detection protocol, the prefix list computation protocol, and the secure left-shift protocol. In Sec. 5, we implement the floating-point multiplication and addition protocols, with some components of the protocol provided in Appendix. A. In Sec. 6, we test our fixed-point and floating-point protocols. Appendix. B contains the security proofs for our protocols. Finally, related work can be found in Appendix C.

2 Preliminaries

Notation. Let $\mathcal{P} := \{P_0, P_1, P_2\}$ be the three MPC parties. We write set $\mathbb{Z}_{2^\ell} := \{0, \dots, 2^\ell - 1\}$ for ℓ -bit ring. For a value $x \in \mathbb{Z}_{2^\ell}$, we write $x_{(i)}$ for its i^{th} bit in big-endian order and $\text{sign}(x)$ for its most significant (sign) bit, namely, $\text{sign}(x) := x_{(0)}$. We define $\text{cut}(x, d)$ as the operation that removes the last d bits of x . Formally, for an element $x \in \mathbb{Z}_{2^\ell}$, it is given by $\text{cut}(x, d) = \lfloor x/2^d \rfloor \pmod{2^{\ell-d}}$. We use $\eta_{j,k}$ to denote the common seed shared by both P_j and P_k . Our protocol employs two types of secret sharing:

Two-party additive secret share. We define a two-party additive secret share of a value $x \in \mathbb{Z}_M$ as $[x]^M := ([x]_1 \in \mathbb{Z}_M, [x]_2 \in \mathbb{Z}_M)$, such that $x = [x]_1 + [x]_2 \pmod{M}$. Party P_j for $j \in \{1, 2\}$ holds the share $[x]_j$.

Three-party masked secret share. We define a three-party masked secret share of a value $x \in \mathbb{Z}_M$ as $\langle x \rangle^M := (m_x, [r_x]^M)$, where $r_x \in \mathbb{Z}_M$ is a fresh random value known to P_0 and $x = m_x + r_x \pmod{M}$. For $j \in \{1, 2\}$, party P_j holds $(m_x \in \mathbb{Z}_M, [r_x]_j \in \mathbb{Z}_M)$, while P_0 holds the additive share $([r_x]_1, [r_x]_2)$.

For notational simplicity, when the modulus M is unambiguous, we abbreviate the secret sharing of x as $[x]$ or $\langle x \rangle$. For a vector $\mathcal{L} = (l_0, \dots, l_{n-1})$ whose elements are secret shared, we hereby define $[\mathcal{L}]$ to be the tuple of component-wise secret sharings, namely, $[\mathcal{L}] = ([l_0], \dots, [l_{n-1}])$. The same convention applies to the $\langle \cdot \rangle$ notation. Both the aforementioned secret-sharing forms have the linear homomorphic

property. In particular, for value x, y and a public constant c , $[x+y] = [x]+[y] = ([x]_1+[y]_1, [x]_2+[y]_2)$, $[c \cdot x] = c \cdot [x] = (c \cdot [x]_1, c \cdot [x]_2)$ and $[x+c] = [x] + c = ([x]_1 + c, [x]_2)$, where all operations are performed modulo M . The same linear operations apply analogously to the $\langle \cdot \rangle$ sharing.

Secret Sharing Scheme. Let $\Pi_{[\cdot]}$ and $\Pi_{\langle \cdot \rangle}$ denote the secret sharing protocols corresponding to the $[\cdot]$ and $\langle \cdot \rangle$ representations, respectively. By $\Pi_{[\cdot]}(x)$ with specified input x , we mean that x is shared by a designated party; by $\Pi_{[\cdot]}$ without an explicit input, we mean the parties jointly generate a secret sharing of a fresh random value. We employ pseudorandom generators (PRG) to reduce the communication [51]. In our protocol description, when we state that parties P_j and P_k jointly sample random values, we mean that they invoke a PRG with the same seed $\eta_{j,k}$. A brief overview of the secret-sharing schemes is given below.

- $[x]^M \leftarrow \Pi_{[\cdot]}^M(x)$: If P_0 holds x , P_0 and P_1 jointly sample a random value $[x]_1 \in \mathbb{Z}_M$ using the seed $\eta_{0,1}$; Party P_0 then sends $[x]_2 = x - [x]_1 \pmod{M}$ to P_2 . If P_1 (resp., P_2) holds the input x , P_1 and P_2 jointly sample a random value $[x]_2 \in \mathbb{Z}_M$; P_1 set $[x]_1 = x - [x]_2 \pmod{M}$.
- $[x]^M \leftarrow \Pi_{[\cdot]}^M$: Parties P_0 and P_1 jointly sample a random value $[x]_1 \in \mathbb{Z}_M$ using the seed $\eta_{0,1}$, and parties P_0 and P_2 jointly sample a random value $[x]_2 \in \mathbb{Z}_M$ using the seed $\eta_{0,2}$.
- $\langle x \rangle^M \leftarrow \Pi_{\langle \cdot \rangle}^M(x, P_k)$: In the offline phase, all parties execute $[r_x]^M \leftarrow \Pi_{[\cdot]}^M$, where P_k knows both seeds corresponding to $[r_x]_1$ and $[r_x]_2$; Party P_k computes $m_x = x - [r_x]_1 - [r_x]_2$ and sends m_x to P_1 and P_2 .
- $\langle x \rangle^M \leftarrow \Pi_{\langle \cdot \rangle}^M$: In the offline phase, all parties execute $[r_x]^M \leftarrow \Pi_{[\cdot]}^M$. Parties P_1 and P_2 then jointly sample a random value m_x using $\eta_{1,2}$.

In our protocol, for the convenience of security proofs, we introduce ideal functionality $\mathcal{F}_{\text{share}}[M]$ to represent the functionality corresponding to $[x]^M \leftarrow \Pi_{[\cdot]}^M$. Specifically, $\mathcal{F}_{\text{share}}$ samples $[x]_1, [x]_2 \xleftarrow{\$} \mathbb{Z}_M$, then sends $[x]_1$ to P_0 and P_1 , and sends $[x]_2$ to P_0 and P_2 .

Preprocessing. We follow the “preprocessing” paradigm [7], which splits the protocol into two phases. The preprocessing (offline) phase is data independent and can be executed before the inputs are known, whereas the online phase is data dependent and is executed after the inputs are provided. Specifically, in our protocols, all random masks r_x associated with sharings $\langle x \rangle$ can be generated in the circuit dependent offline phase. During the online phase, the parties only need to collaboratively compute the corresponding masked values m_x and deliver them to P_1 and P_2 .

Multiplication Over Ring. We use Π_{mult}^M to denote multiplication over the ring $\mathbb{Z}_M$. For sharings of the form $\langle \cdot \rangle$, we adopt the protocol proposed in ASTRA [12], which is secure in the semi-honest setting. For the multiplication $\langle z \rangle^M \leftarrow \Pi_{\text{mult}}^M(\langle x \rangle^M, \langle y \rangle^M)$ corresponding to $z = x \cdot y$, all parties first generate $[r_z] \leftarrow \Pi_{[\cdot]}$ for the output wire in the offline phase. In the online phase, the parties compute the masked value m_z , which can be written as

$$\begin{aligned} m_z &= xy - r_z = (m_x + r_x)(m_y + r_y) - r_z \\ &= m_x m_y + m_x r_y + m_y r_x + r_x r_y - r_z. \end{aligned}$$

The term $[\Gamma']^M = m_x m_y + m_x [r_y]^M + m_y [r_x]^M$ can be computed by P_1 and P_2 locally, while $[\Gamma]^M = [r_x \cdot r_y]^M - [r_z]^M$ can be secret shared by P_0 to P_1 and P_2 in the offline phase. In the online phase, P_1 and P_2 compute and reconstruct $[m_z]^M = [\Gamma']^M + [\Gamma]^M$.

For $x_1 \in \mathbb{Z}_M$ input by P_1 and $x_2 \in \mathbb{Z}_M$ input by P_2 , we abuse the notation $[z]^M \leftarrow \Pi_{\text{mult}}^M(x_1, x_2)$ to denote the multiplication of x_1 and x_2 , yielding a secret-shared result in the $[\cdot]$. In the offline phase, for $i \in \{1, 2\}$, P_0 and P_i jointly generate a random value r_i using same seed. Party P_0 computes $r = r_1 \cdot r_2$ and invokes $\Pi_{[\cdot]}(r)$ to secret share r to P_1 and P_2 . In the online phase, each party P_i , for $i \in \{1, 2\}$, sends $a_i = x_i - r_i$ to P_{3-i} . Then P_1 sets $[z]_1^M = a_1 \cdot a_2 + a_2 \cdot r_1 + [r]_1^M$ and P_2 sets $[z]_2^M = a_1 \cdot r_2 + [r]_2^M$.

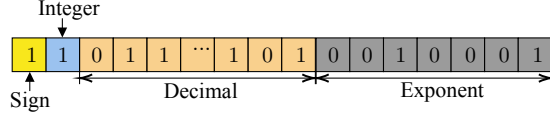


Figure 1: Our Floating-point Structure.

Fixed-point Representation. For a real number x with one sign-bit, $\ell_i - 1$ bits of integer precision and ℓ_f bits of fractional precision, we define its fixed-point representation (over ring) as $\hat{x} = \lfloor x \cdot 2^{\ell_f} \rfloor \pmod{2^\ell}$, while $\ell_i + \ell_f = \ell$. This encoding designates the most significant bit as the sign bit. Negative values are encoded into the interval $(2^\ell - 2^{\ell-2}, 2^\ell)$, while positive values are encoded into $[0, 2^{\ell-2}]$. Compared to conventional fixed-point representation [36, 34] and [43, 21, 24], this encoding sacrifices one bit of precision. In Section 3, we explain the rationale behind adopting this representation method. Under this representation, the addition of two fixed-point numbers can be directly implemented using ring addition. Specifically, for $z = x + y$, we have $\hat{z} = \hat{x} + \hat{y} \pmod{2^\ell}$. However, multiplication cannot be implemented directly using the ring multiplication operation. Since $\hat{x} = x \cdot 2^{\ell_f}$, the product $\hat{x} \cdot \hat{y}$ introduces a scaling factor of $2^{2\ell_f}$. The traditional approach addresses this issue via truncation [18], namely by computing $\hat{z} = \hat{x} \cdot \hat{y} / 2^{\ell_f}$. This approach, however, requires computations to be performed on larger rings. In particular, performing the multiplication $\hat{x} \cdot \hat{y}$ over an ℓ -bit ring causes overflow and results in the loss of higher-order bits. To guarantee correctness, the multiplication must be carried out over at least an $\ell + \ell_f$ -bit ring.

Truncation. Truncation is a costly computation in MPC, with the most efficient known approach being probabilistic truncation [53]. Probabilistic truncation is directly achieved by locally right-shifting the secret shares by the corresponding number of bits. However, this technique introduces a non-negligible probability of high-order truncation errors. Suppose that the effective bit length of x is ℓ' , and the secret sharing is performed on a ring of ℓ bits. Then the probability of this error occurring is $2^{\ell' - \ell - 1}$ (see Zhou et al. [53]). To mitigate this error, it is necessary to reserve an additional ℓ_a bits set to 0, beyond those required to avoid arithmetic overflow. With this padding, the probability of a truncation error is reduced to $2^{-\ell_a - 1}$. To make the error probability negligible, ℓ_a is typically chosen to be larger than 32. Consequently, existing fixed-point computations generally require computations to be carried out over a ring of size at least $\ell + \ell_f + \ell_a$ bits.

Floating-point Representation. Floating-point numbers provide a significantly larger dynamic range—often by orders of magnitude—than fixed-point representations. For a real number x , we denote its floating-point representation (over ring) by $\bar{x}$. Fig. 1 illustrates our floating-point structure. We define $\bar{x} := (b_x, \hat{x}, \tilde{x})$, where b_x is the sign bit, $\hat{x}$ is the fixed-point mantissa, and $\tilde{x}$ is the exponent. The mantissa $\hat{x}$ consists of a one-bit integer (fixed to 1 in normalized form) and an ℓ_f -bit fractional part. The exponent $\tilde{x}$ has ℓ_e bits used to represent a number in the range $[-2^{\ell_e-1}, 2^{\ell_e-1}]$. We let it add a bias 2^{ℓ_e-1} to map it into the interval $[0, 2^{\ell_e} - 1]$. Under this encoding, the real value x is interpreted as $x = (-1)^{b_x} \cdot \hat{x} \cdot 2^{\tilde{x} - 2^{\ell_e-1}}$.

3 Succinct Fixed Point Multiplication

In this section, we present a succinct fixed-point multiplication protocol, whose online cost requires only 2ℓ bits for multiplying values represented with ℓ_f fractional bits and ℓ_i integer bits, while $\ell_f + \ell_i = \ell - 1$.

Intuition. We design a protocol that performs a $\ell - 1$ -bit fixed-point multiplication only over an ℓ -bit ring, rather than $\ell + \ell_f + \ell_a$ -bit. Let x and y be two values represented with $\ell - 1$ -bit precision, consisting of ℓ_f fractional bits and ℓ_i integer bits, where $\ell_f + \ell_i = \ell - 1$. Their corresponding fixed-point encoding (over ring) is $\hat{x} = \lfloor x \cdot 2^{\ell_f} \rfloor$ and $\hat{y} = \lfloor y \cdot 2^{\ell_f} \rfloor$. Fixed-point multiplication is defined as

$$\hat{z} = \text{cut}(\hat{x} \cdot \hat{y} \pmod{2^{\ell_f + \ell}}, \ell_f),$$

which satisfies $\hat{z} = z \cdot 2^{\ell_f}$, where $z = x \cdot y$.

Protocol $\langle z \rangle^{2^{\ell'}} \leftarrow \Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}(\langle x \rangle^{2^\ell}, \langle y \rangle^{2^\ell})$

P_j and P_k hold the common seed $\eta_{j,k} \in \{0, 1\}^\lambda$.

Input : $\langle \cdot \rangle^{2^\ell}$ -shared value of x and y .

Output : $\langle \cdot \rangle^{2^{\ell'}}$ -shared value of $z = x \cdot y$.

Offline:

- P_0 calculates $r_{x,(0)}, r_{y,(0)}, w_x^{(0)} = 1\{[r_x]_0 + [r_x]_1 \stackrel{?}{\neq} r_x\}$ and $w_y^{(0)} = 1\{[r_y]_0 + [r_y]_1 \stackrel{?}{\neq} r_y\}$;
- P_0 calculates $\gamma_y = w_y^{(0)} + r_{y,(0)} \pmod{2^{\ell_f + \ell' - \ell}}$ and $\gamma_x = w_x^{(0)} + r_{x,(0)} \pmod{2^{\ell_f + \ell' - \ell}}$;
- P_0 calculates $\beta_x = (1 - r_{y,(0)})([r_x]_1 + [r_x]_2) \pmod{2^{\ell_f + \ell' - \ell}}$ and $\beta_y = (1 - r_{x,(0)})([r_y]_1 + [r_y]_2) \pmod{2^{\ell_f + \ell' - \ell}}$;
- P_0 calculates $\Gamma = 2^\ell((w_y^{(0)} + r_y[0])([r_x]_1 + [r_x]_2) + (w_x^{(0)} + r_x[0])([r_y]_1 + [r_y]_2)) + ([r_x]_1 + [r_x]_2)([r_y]_1 + [r_y]_2) \pmod{2^{\ell' + \ell_f}}$;
- P_0 performs $[r_{x,(0)}], [r_{y,(0)}], [\gamma_x], [\gamma_y], [\beta_x], [\beta_y] \leftarrow \Pi_{[\cdot]}^{\ell_f + \ell' - \ell}(r_x[0], r_y[0], \gamma_x, \gamma_y, \beta_x, \beta_y)$ and $[\Gamma] \leftarrow \Pi_{[\cdot]}^{\ell'}(\Gamma)$;
- All parties invoke $[r_z] \leftarrow \mathcal{F}_{\text{share}}[2^{\ell'}]$

Online:

- P_j , for $j \in \{1, 2\}$ does:
 1. Compute $m_x = m_x + 2^{\ell-2} \pmod{2^\ell}$ and $m_y = m_y + 2^{\ell-2} \pmod{2^\ell}$;
 2. Calculate $[z] = m_x m_y + m_x[r_y] + m_y[r_x] + [\Gamma] + 2^\ell(m_{y,(0)}m_x(1 - [r_{y,(0)}]) + m_{y,(0)}[\beta_x] + m_x[\gamma_y] + m_x[0]m_y(1 - [r_{x,(0)}]) + m_x[0][\beta_y] + m_y[\gamma_x]) + 2^{2\ell-2}([\gamma_y] + m_{y,(0)} - m_{y,(0)}[r_{y,(0)}] + [\gamma_x] + m_{x,(0)} - m_{x,(0)}[r_{x,(0)}]) + 2^{\ell-2}(m_x + m_y + [r_x] + [r_y]) \pmod{2^{\ell' + \ell_f}}$;
 3. Drop the lower ℓ_f bits of $[z]^{\ell' + \ell_f}$ to get $[z]^{\ell'}$;
 4. Reveal $[z] - [r_z]$ to get m_z ;

Figure 2: Fixed-point Multiplication Protocol.

Our goal is to compute $\langle \hat{z} \rangle^{2^\ell}$ directly from $\langle \hat{x} \rangle^{2^\ell}$ and $\langle \hat{y} \rangle^{2^\ell}$, while performing all operations over an ℓ -bit ring. The key idea is to reinterpret the sharings $\langle \hat{x} \rangle^{2^\ell}$ and $\langle \hat{y} \rangle^{2^\ell}$ as sharings $\langle \hat{x}' \rangle^{2^{\ell+\ell_f}}$ and $\langle \hat{y}' \rangle^{2^{\ell+\ell_f}}$, and then carry out the multiplication over the $\ell + \ell_f$ -bit ring. However, this naive reinterpretation yields an incorrect result. This error arises because directly lifting an ℓ -bit sharing to an $\ell + \ell_f$ -bit ring causes the wrap-around bit to become significant, thereby corrupting the result. Our key observation is that this error can be identified and eliminated during the online phase with zero additional overhead. Let w_x and w_y denote the wrap-around values of the sharings $\langle \hat{x} \rangle^{2^\ell}$ and $\langle \hat{y} \rangle^{2^\ell}$, respectively. That is, $m_{\hat{x}} + [r_{\hat{x}}]_1 + [r_{\hat{x}}]_2 = \hat{x} + 2^\ell \cdot w_x$, $w_x \in \{0, 1, 2\}$, and similarly for w_y . It holds that $\hat{x}' = \hat{x} + 2^\ell \cdot w_x$. The computed product is therefore $(\hat{x} + 2^\ell w_x)(\hat{y} + 2^\ell w_y) \pmod{2^{\ell+\ell_f}}$, whose error term is $2^\ell(w_y \hat{x} + w_x \hat{y}) + 2^{2\ell} w_x w_y$. By subtracting this error, we can recover the correct product $\hat{x} \cdot \hat{y}$. With this correction, the product $\hat{x} \hat{y}$ lies within $\ell + \ell_f$ bits, allowing the lower ℓ_f bits to be directly discarded, avoiding the truncation issues encountered in prior approaches [45, 34] and [11]. At a high level, our protocol consists of the following three steps.

- Interpret $\langle x \rangle^{2^\ell}$ and $\langle y \rangle^{2^\ell}$ as sharings over a $\ell + \ell_f$ -bit ring and perform multiplication.
- Subtract the error term $2^\ell(w_y \hat{x} + w_x \hat{y}) + 2^{2\ell} w_x w_y$.
- Apply cut to obtain the final ℓ -bit result.

We observe that the three steps above can be completed within a single round, resulting in an online communication cost of only 2ℓ bits, which matches the cost of a single ℓ -bit multiplication.

Wrapping Bit Calculation. Traditional protocols for computing the wrap bit are highly complex [32], incurring costs comparable to secure comparison and typically exceeding the cost of multiplication by more than an order of magnitude. However, the wrap bit is closely related to the most significant bit (MSB). Given the MSB, the wrap bit can be computed very efficiently for two-party additive secret sharing [16, 28].

Considering $x = x_0 + x_1 \pmod{2^\ell}$ and the first bit of x is zero ($x \in [0, 2^{\ell-1})$), the wrap-bit b (it holds that $x_0 + x_1 = x + b \cdot 2^\ell$) can be calculated by $b = x_{0,(0)} \vee x_{1,(0)}$, while $x_{0,(0)}$ and $x_{1,(0)}$ denote the most significant bits of x_0 and x_1 , respectively. The correctness follows from a simple case analysis. Since the most significant bit of x is 0: (i) if $x_{0,(0)} = 0$ and $x_{1,(0)} = 1$, then a carry from lower bits must occur to produce $x_{0,(0)} = 0$, implying $b = 1$; (ii) if $x_{0,(0)} = 0$ and $x_{1,(0)} = 0$, no carry arises and $b = 0$; (iii) if $x_0[0] = 1$ and $x_1[0] = 1$, their sum produces a carry regardless of lower bits, and thus $b = 1$. This shows that the wrap bit can be efficiently computed from the most significant bits of the shares when the MSB of the secret is known.

Based on above observation, consider shared inputs $\langle \hat{x} \rangle^{2^\ell}$ and $\langle \hat{y} \rangle^{2^\ell}$, where the effective bit length of $\hat{x}$ and $\hat{y}$ is $\ell-1$. That is, $\hat{x} \in [0, 2^{\ell-2}) \cup [2^{\ell-2}, 2^\ell)$, and similarly for $\hat{y}$. To facilitate error computation, we first ensure that their most significant bits are zero by shifting both values: $\langle \hat{x}' \rangle = \langle \hat{x} \rangle + 2^{\ell-2}$ and $\langle \hat{y}' \rangle = \langle \hat{y} \rangle + 2^{\ell-2}$. We then reinterpret these sharings over a larger ring by directly setting $\langle \hat{x}' \rangle^{2^{\ell_f+\ell}} := \langle \hat{x}' \rangle^{2^\ell}$ for $i \in \{0, 1, 2\}$, and analogously for $\hat{y}'$. It follows that $\hat{x}' = \hat{x} + 2^{\ell-2} + 2^\ell \cdot w_x \pmod{2^{\ell_f+\ell}}$, where w_x is the wrap value of $\hat{x}'$, and similarly for w_y . Thus,

$$\hat{x}\hat{y} = (\hat{x}' - 2^\ell w_x - 2^{\ell-2})(\hat{y}' - 2^\ell w_y - 2^{\ell-2}) \pmod{2^{\ell_f+\ell}}.$$

The wrap value can be decomposed as $w_{\hat{x}} = w_{\hat{x}}^{(0)} + w_{\hat{x}}^{(1)}$, where $w_{\hat{x}}^{(0)} = (r_{\hat{x}'} \stackrel{?}{\neq} [r_{\hat{x}'}]_0 + [r_{\hat{x}'}]_1)$ is the wrap-bit of $[r_{\hat{x}'}]_0 + [r_{\hat{x}'}]_1$, $w_{\hat{x}}^{(1)} = (\hat{x} \stackrel{?}{\neq} r_{\hat{x}'} + m_{\hat{x}'})$ is the wrap-bit of $m_{\hat{x}'} + r_{\hat{x}'}$.

Expanding the product yields,

$$\begin{aligned} \hat{x}\hat{y} = & \hat{x}'\hat{y}' - 2^\ell(w_y\hat{x}' + w_x\hat{y}') + 2^{2\ell-2}(w_x + w_y) + 2^{2\ell-4} \\ & - 2^{\ell-2}(\hat{x}' + \hat{y}') + 2^{2\ell}w_xw_y \pmod{2^{\ell_f+\ell}}. \end{aligned} \quad (1)$$

Since $2^{2\ell}w_xw_y \pmod{2^{\ell_f+\ell}} \equiv 0$, this term can be safely ignored. We observe that the remaining expression can be written as a linear combination of terms involving masked values (m -related terms, e.g., $m_{\hat{x}'}$ or $m_{\hat{y}'}$), with coefficients derived solely from random-mask-related terms (r -related terms, e.g., $[r_{\hat{x}'}]_1$ or $[r_{\hat{y}'}]_2$). As party P_0 knows the r -related terms, while parties P_1 and P_2 know the m -related terms, we let P_0 compute and secret-share the coefficients during the offline phase. As the m -related terms are public to P_1 and P_2 , in the online phase, they only need to locally multiply these secret-shared coefficients with the corresponding masked values to obtain the final secret sharing $[\hat{x} \cdot \hat{y}]$.

For $\hat{x}'\hat{y}'$, we have

$$\begin{aligned} \hat{x}'\hat{y}' = & m_{\hat{x}'}m_{\hat{y}'} + m_{\hat{x}'}([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2) + m_{\hat{y}'}([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2) \\ & + ([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2) \pmod{2^{\ell_f+\ell}} \end{aligned}$$

If P_0 computes $\gamma = ([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2) \pmod{2^{\ell_f+\ell}}$ and secret share it to P_1 and P_2 , then P_1 and P_2 can directly compute the shared value $[\hat{x}'\hat{y}']$ over ring $2^{\ell_f+\ell}$. Specifically,

- P_1 computes $[\hat{x}'\hat{y}']_1 = m_{\hat{x}'}[r_{\hat{y}'}]_1 + m_{\hat{y}'}[r_{\hat{x}'}]_1 + [\gamma]_1$;
- P_2 computes $[\hat{x}'\hat{y}']_2 = m_{\hat{x}'}m_{\hat{y}'} + m_{\hat{x}'}[r_{\hat{y}'}]_2 + m_{\hat{y}'}[r_{\hat{x}'}]_2 + [\gamma]_2$.

For $2^\ell w_y\hat{x}'$, we have

$$\begin{aligned} w_y\hat{x}' = & (w_y^{(0)} + m_{\hat{y}',(0)} + r_{\hat{y}',(0)} - m_{\hat{y}',(0)}r_{\hat{y}',(0)})\hat{x}' \\ = & m_{\hat{y}',(0)}(1 - r_{\hat{y}',(0)})m_{\hat{x}'} + (w_y^{(0)} + r_{\hat{y}',(0)})m_{\hat{x}'} \\ & + m_{\hat{y}',(0)}(1 - r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2) \\ & + (w_y^{(0)} + r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2) \pmod{2^{\ell_f+\ell}} \end{aligned}$$

Functionality $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$

- $\mathcal{F}_{\text{fix-mult}}$ receives shares of x and y from all parties;
- $\mathcal{F}_{\text{fix-mult}}$ reconstructs x and y , then performs $z = x \cdot y$; Let z_f denote the last ℓ_f bits of z and $\hat{z}$ denote the result after truncating the least ℓ_f bits. The function $\mathcal{F}$ randomly samples z' based on z_f : with probability $(z_f + 1)/2^{\ell_f}$, $z' = \hat{z}$; and with probability $1 - (z_f + 1)/2^{\ell_f}$, $z' = \hat{z} - 1$;
- $\mathcal{F}_{\text{fix-mult}}$ generates $\langle \cdot \rangle^{2^{\ell'}}$ -share of z' , and distributes $\langle z' \rangle^{2^{\ell'}}$ to all parties.

Figure 3: Functionality of Fixed-point Multiplication with Probabilistic Truncation.

In this expression, $r_{\hat{y}',(0)}$ denotes the bit of $r_{\hat{y}'}$ at position 0. If P_0 computes the values $r_{\hat{y}',(0)}$, $w_y^{(0)} + r_{\hat{y}',(0)}$, $(1 - r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)$, and $(w_y^{(0)} + r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)$, and secret shares these to P_1 and P_2 , then P_1 and P_2 can directly compute the shared value $[w_y \hat{x}']$ over ring $2^{\ell_f + \ell}$. A similar method can be used to compute $[w_x \hat{y}']$.

For $2^{\ell-2}(\hat{x}' + \hat{y}')$, we have

$$\begin{aligned} \hat{x}' + \hat{y}' = & m_{\hat{y}'} + m_{\hat{x}'} + \\ & [r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2 + [r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2 \pmod{2^{\ell_f + \ell}} \end{aligned}$$

It can be directly calculated by P_1 and P_2 .

For $2^{\ell-2}(w_x + w_y)$, we have

$$w_y := w_y^{(0)} + m_{\hat{y}',(0)} + r_{\hat{y}',(0)} - m_{\hat{y}',(0)} r_{\hat{y}',(0)}$$

Since $r_{\hat{y}',(0)}$, $w_y^{(0)} + r_{\hat{y}',(0)}$ have already been shared by P_0 when calculating $w_y \hat{x}'$, the value $[w_x]$ and $[w_y]$ can be directly computed by P_1 and P_2 .

Put all together, in the offline phase, P_0 needs to secretly share the following values to P_1 and P_2 : $r_{\hat{y}',(0)}$, $w_y^{(0)} + r_{\hat{y}',(0)}$, $(1 - r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)$, $r_{\hat{x}',(0)}$, $w_x^{(0)} + r_{\hat{x}',(0)}$, $(1 - r_{\hat{x}',(0)})([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2)$. These shared values will allow P_1 and P_2 to locally multiply the m -related terms during the online phase. For the constant terms that do not need to be multiplied by m -related terms, P_0 combines them into a single value $\Gamma = -2^\ell(w_y^{(0)} + r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2) - 2^\ell(w_x^{(0)} + r_{\hat{x}',(0)})([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2) + ([r_{\hat{y}'}]_1 + [r_{\hat{y}'}]_2)([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)$ and secretly shares this value as well. In the online phase, P_1 and P_2 locally compute the linear combination of the shared term to obtain the two-party secret share $[\hat{x}\hat{y}]$ over $\mathbb{Z}_{2^{\ell_f + \ell}}$. Since it holds that $m_{\hat{z}} = \text{cut}(\hat{x}\hat{y}, \ell_f) - r_{\hat{z}} \pmod{2^\ell}$, P_1 and P_2 then drop the lower ℓ_f bits of $[\hat{x}\hat{y}]$ locally, subtract the pre-generated random value $r_{\hat{z}}$ for the output, and finally open the result to obtain $m_{\hat{z}}$.

Communication Optimization and Generalization. It is easy to see that, in our proposed scheme, the online phase is succinct: for an $\ell - 1$ -bit fixed-point number, only a single round of communication with an opening cost of 2ℓ bits is required. In the offline phase, 7 elements over $\mathbb{Z}_{2^{\ell_f + \ell}}$ need to be transmitted.

We observe that terms such as $r_{\hat{y}',(0)}$, $w_y^{(0)} + r_{\hat{y}',(0)}$ and $(1 - r_{\hat{y}',(0)})([r_{\hat{x}'}]_1 + [r_{\hat{x}'}]_2)$ do not need to be secret-shared over the large ring $\mathbb{Z}_{2^{\ell_f + \ell}}$. Since computing $2^\ell w_y \hat{x}'$ contains a scale of 2^ℓ , the above terms can directly shared over ring $\mathbb{Z}_{2^{\ell_f}}$ and later treated as share over $\mathbb{Z}_{2^{\ell_f + \ell}}$ during computations involving terms such as $2^\ell m_{\hat{x}'} r_{\hat{y}',(0)}$ or $[r_{\hat{y}',(0)}]^{2^{\ell_f}}$, without affecting the correctness. The reason is that the wrap-bit error b from interpreting $[r_{\hat{y}',(0)}]_1^{2^{\ell_f}} + [r_{\hat{y}',(0)}]_2^{2^{\ell_f}} = b \cdot 2^{\ell_f} + r_{\hat{y}',(0)}$ as a sharing over $\mathbb{Z}_{2^{\ell_f + \ell}}$ is multiplied by 2^ℓ , thereby restoring it to a wrap-bit. Consequently, it suffices to secret share only $r_{\hat{y}',(0)}$ and $2^\ell(w_y^{(0)} + r_{\hat{y}',(0)})$ over $\mathbb{Z}_{2^{\ell_f}}$. During the online phase, P_1 and P_2 can simply view them as share over $\mathbb{Z}_{2^{\ell_f + \ell}}$ for further computation.

Our protocol naturally supports output generation over larger rings, as it relies on local ring extension followed by a cut operation. Fig. 2 presents a generalized version of the protocol, which takes ℓ -bit

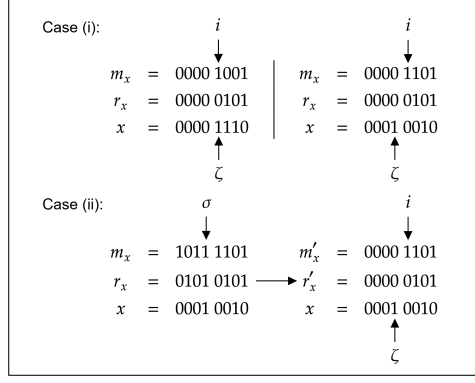


Figure 4: First Non-Zero Bit Detection

secret-shared inputs and produces ℓ' -bit secret-shared outputs. When $\ell' = \ell$, it represents the standard fixed-point multiplication case. Considering inputs with an ℓ_i -bit integer part and an ℓ_f -bit fractional part, the multiplication result can have up to $2\ell_i$ bits in the integer part. Therefore, we can set ℓ' to $\ell + \ell_i$ to preserve all potential overflow bits. In this case, Γ needs to be secret-shared over a ring of size $\ell' + \ell_f$ bits. Meanwhile, the aforementioned values such as $r_{x,(0)}$ must be secret-shared over a ring of $\ell_f + \ell' - \ell$ bits to ensure that after multiplying by 2^ℓ , the wrap-bit is correctly restored to the wrap position. Notably, due to the above protocol, we have $2^{2\ell}w_xw_y \bmod 2^{\ell_f+\ell} = 0$, allowing us to ignore the term $2^{2\ell}w_xw_y$. However, when we require an ℓ' -bit output, $2^{2\ell}w_xw_y \bmod 2^{\ell_f+\ell'}$ is not necessarily zero. To ignore this term, we must satisfy $\ell' \leq 2\ell - \ell_f$. If we need to handle cases where $\ell' > 2\ell - \ell_f$, additional r -related terms for calculating w_xw_y must be introduced in the offline phase. Consequently, the online phase requires only ℓ' bits of communication cost, and in the offline phase amounts to $6(\ell_f + \ell' - \ell) + \ell' + \ell_f$ bits. If we ignore the least significant bit of precision, P_0 can directly send the cut (ℓ' -bit) constant term Γ , which further reduces the cost to $6(\ell_f + \ell' - \ell) + \ell'$ bits.

Security. Since the final cut operation in our protocol is probabilistic—namely, the least significant bit of the result may be reduced by one with a certain probability—the corresponding ideal functionality must capture this behavior. Fig. 3 illustrates the functionality of Fixed-point Multiplication with Probabilistic Truncation, which probabilistically determines whether a one-bit carry reduction is needed based on the size of the discarded portion. Our protocol securely computes this functionality, and the theorem is as follows. It is worth noting that Li et al. [27] proved that some probabilistic truncation protocols have security issues under a simulation-based security model, but our scheme does not face this problem.

Theorem 1. *The protocol $\Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}$ as depicted in Fig. 2 securely computes $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$ in the $\mathcal{F}_{\text{share}}$ -hybrid model against semi-honest PPT adversaries who can statically corrupt up to one party.*

Proof. Cf. Appendix. B.1 □

4 Constant-round Secure Alignment

In this section, we propose a novel constant-round secure alignment protocol Π_{align} . For a value x whose first non-zero bit (from the most significant bit, i.e., in big-endian order) is located at position ζ , the protocol $(\langle z \rangle, \langle \delta \rangle) \leftarrow \Pi_{\text{align}}(\langle x \rangle)$ outputs an aligned sharing $\langle z \rangle$, where $z = 2^\zeta \cdot x$. That is, the protocol left-shifts x by ζ positions so that the most significant bit of z is equal to 1. The protocol consists of two steps: securely detecting the position δ of the first non-zero bit of x ; then left shifting x by ζ positions.

Intuition. We observe that when $x = m_x + r_x$, the index ζ of the first non-zero bit of x falls into two cases:

- (i) If the wrap bit $b = 0$ (for $x = m_x + r_x \pmod{2^\ell}$), we have $(m_{x,(i)}, r_{x,(i)}) = (0, 0)$ for all $i < \zeta$. Let i be the first index where $(m_{x,(i)}, r_{x,(i)}) \in (1, 0), (0, 1), (1, 1)$. If no carry is generated from less significant positions, then $\zeta = i$; otherwise $\zeta = i - 1$;

- (ii) If the wrap bit $b = 1$, then we can find a σ that for all $i < \sigma$, we have $(m_{x,(i)}, r_{x,(i)}) \in (1, 0), (0, 1)$, and at $i = \sigma$ we have $(m_{x,(\sigma)}, r_{x,(\sigma)}) = (1, 1)$. Zeroing all positions i up to and including σ , namely, setting $(m_{x,(i)}, r_{x,(i)}) = (0, 0)$ for $i \in [0, \sigma]$, reduces the pattern to case (i). (This is because the operation sets the wrap bit to zero without changing the value x .)

Based on this observation, for the case $b = 0$, if we can detect the first position ζ' in which $m_{x,(\zeta')} \vee r_{x,(\zeta')} = 1$ (corresponding to smallest i for $(m_{x,(i)}, r_{x,(i)}) \in (1, 0), (0, 1), (1, 1)$), we can conclude that the position of the first non-zero bit is either $\zeta' - 1$ or ζ' . For the case $b = 1$, we first locate the earliest index ζ' such that $m_{x,(\zeta')} \wedge r_{x,(\zeta')} = 1$ (corresponding to smallest i for $(m_{x,(\sigma)}, r_{x,(\sigma)}) = (1, 1)$). We then zero out the prefix up to ζ' and rerun the above $b = 0$ procedure to obtain ζ'' . Finally, we select ζ from ζ', ζ'' using b (i.e., output $\zeta = \zeta'$ if $b = 1$, and $\zeta = \zeta''$ if $b = 0$). We then extract the bit at position $\zeta - 1$; if it is 1, output $\zeta - 1$; otherwise, output ζ . Fig. 4(See Appendix) shows an example.

Theorem 2. Let $x = m + r \pmod{2^\ell}$ with the binary representation $x = x_{(0)} \| x_{(1)} \| \dots \| x_{(\ell-1)}$ ($x_{(0)}$ is the most significant bit) be a non-zero value, and let ζ be the position of the first non-zero bit of x , i.e.,

$$x_{(\zeta)} = 1 \wedge \forall i < \zeta : x_{(i)} = 0$$

Let $b \in \{0, 1\}$ be the wrap-bit such that $m + r = 2^\ell \cdot b + x$. If $b = 1$, there exists an index σ such that $m_{(i)} \wedge r_{(i)} = 0$ for $i < \sigma$ and $m_{(\sigma)} \wedge r_{(\sigma)} = 1$. If we define m' and r' by setting $m'_{(i)} = 0, r'_{(i)} = 0$ for all $i \leq \sigma$ (and $m'_{(i)} = m_{(i)}, r'_{(i)} = r_{(i)}$ for all $i > \sigma$), then $m' + r' = x$ and the corresponding wrap-bit $b' = 0$. If $b = 0$, there exists an index σ such that $m_{(\sigma)} \vee r_{(\sigma)} = 1$ and $m_{(i)} \vee r_{(i)} = 0$ for $i < \sigma$. Then ζ can only be either σ or $\sigma - 1$.

Proof. Cf. Appendix. B.2 □

At a high level, our secure alignment protocol consists of three subprotocols:

- We propose a first non-zero element detection protocol Π_{detect} , while it receives an input list $([l_0], \dots, [l_{\ell-1}])$ and outputs a shared index $[\sigma]$. It holds that $l_\sigma = 1$ and $l_i = 0$ for $i < \sigma$. For the case $b = 0$, we first calculate $m_{x,(i)} \vee r_{x,(i)}$ for each bit i . Note that $m_{x,(i)} \vee r_{x,(i)} = m_{x,(i)} + r_{x,(i)} - m_{x,(i)} \cdot r_{x,(i)}$. When P_0 secret shares $[r_{x,(i)}]$ to P_1 and P_2 , P_1 and P_2 , who known $m_{x,(i)}$, can locally compute the corresponding secret shares $[m_{x,(i)} \vee r_{x,(i)}]$. We denote the resulting list as $[\mathcal{L}_1]$. Next, we invoke the first non-zero item detection protocol Π_{detect} on $[\mathcal{L}_1]$ to obtain the shared position $[\zeta_1]$. As discussed earlier, ζ_1 corresponds to the first position where $(1, 0)/(0, 1)/(1, 1)$.
- For the case $b = 1$, our goal is to construct another list $[\mathcal{L}'_1]$ by zeroing out the prefix of $\mathcal{L}_1$ up to the first position where $(m_x[i], r_x[i]) = (1, 1)$. We compute $m_x[i] \wedge r_x[i]$ for each bit i . Since $m_{x,(i)} \wedge r_{x,(i)} = m_{x,(i)} \cdot r_{x,(i)}$, this can be done locally by P_1 and P_2 once P_0 has secret-shared $[r_{x,(i)}]$. We denote the resulting list as $[\mathcal{L}_2]$. We then invoke Π_{detect} on $[\mathcal{L}_2]$ to obtain the shared index $[\zeta_2]$, corresponding to the first position where $(m_x[i], r_x[i]) = (1, 1)$. Note that, in the case $b = 1$, the list $\mathcal{L}_1$ consists of all 1 prefixes up to ζ_2 . Therefore, we can subtract 1 from the first ζ_2 entries of $[\mathcal{L}_1]$ to obtain the modified list $[\mathcal{L}'_1]$, in which the prefix up to ζ_2 is zeroed out. Π_{prefix} converts the index ζ_2 into a prefix indicator list

$$\mathcal{D} = (\underbrace{1, 1, \dots, 1}_{\zeta_2}, \underbrace{0, 0, \dots, 0}_{\ell - \zeta_2}).$$

We have $\mathcal{L}'_1 = \mathcal{L}_1 - \mathcal{D}$. We use the prefix list calculation protocol Π_{prefix} to denote the above process of deriving list $\mathcal{D}$ from list $\mathcal{L}_2$. To ensure that Π_{detect} always operates the correct list $[\mathcal{L}_1]$ regardless of whether $b = 0$ or $b = 1$, we use the control bit (corresponding to wrap-bit) b to select between $\mathcal{D}$ and an all-zero list. When $b = 0$, the resulting $\mathcal{D}$ is set to an all-zero list. It can be observed that when $b = 0$, $\mathcal{L}_1 - \mathcal{D}$ remains the same as the original $\mathcal{L}_1$; when $b = 1$, $\mathcal{L}_1 - \mathcal{D}$ removes the front ones, thereby transforming the second case into the first one. Formally, the protocol Π_{prefix} takes as input a

Protocol $[\zeta]^K \leftarrow \Pi_{\text{detect}}^{p,K,\ell}([l_0]^p, \dots, [l_{\ell-1}]^p)$

Input : Shared list $[\mathcal{L}]^p := ([l_0]^p, \dots, [l_{\ell-1}]^p)$.

Output : Shared value $[\zeta]^K$, while $l_\zeta = 1$ for the smallest ζ .

Execution:

- P_j , for $j \in \{1, 2\}$, locally calculates $[l'_i] = (\sum_{t=0}^i [l_t]) - 2 \cdot [l_i] + 1 \bmod p$, for $i \in \mathbb{Z}_\ell$.
- P_j , for $j \in \{1, 2\}$, picks same random value $w_i \in \mathbb{Z}_p^*$ and random permutation $\pi : \mathbb{Z}_\ell \mapsto \mathbb{Z}_\ell$ with same seed;
- P_j , for $j \in \{1, 2\}$, calculates $([l''_0], \dots, [l''_{\ell-1}]) = \pi(w_0 \cdot [l'_0], \dots, w_{\ell-1} \cdot [l'_{\ell-1}])$;
- P_j , for $j \in \{1, 2\}$, reveals $(l''_0, \dots, l''_{\ell-1})$ to P_0 ;
- P_0 find out ζ' where $l''_{\zeta'} = 0$;
- P_0 construct list $s := (s_0, \dots, s_{\ell-1})$ where $s'_{\zeta} = 1$ and $s_i = 0$ for $i \neq \zeta'$;
- All parties invoke $[s]^K \leftarrow \Pi_{[\cdot]}^K(s)$;
- P_1 and P_2 calculates $[\zeta]^K = \sum_{i=0}^{\ell-1} \pi^{-1}(i) \cdot [s_i]^K$.

Figure 5: First Non-zero Element Detection Protocol Π_{detect} .

shared binary list $[\mathcal{L}_2]$ and a shared control bit b , and outputs a new shared binary list $[\mathcal{D}]$. When $b = 1$, all elements of $\mathcal{D}$ up to the first non-zero element of $\mathcal{L}_2$ are set to 1, and the remaining elements are set to 0; when $b = 0$, all elements of $\mathcal{D}$ are set to 0. After computing $\mathcal{L}_1 = \mathcal{L}_1 - \mathcal{D}$, we invoke Π_{detect} on $\mathcal{L}_1$ to obtain the shared position $[\zeta_1]$.

- We compute the wrap bit b using the wrap-bit detection protocol Π_{wrap1} . Since the first non-zero bit is in the position either $\zeta_1 - 1$ or ζ_1 , we design a left-shift protocol Π_{shift} to shift x by $\zeta_1 - 1$ positions. Then, we employ the sign bit detection Π_{sign} to check whether the most significant bit of the shifted value is zero. If it is nonzero, this indicates that the first nonzero bit is located at position ζ_1 , while we further left-shift x by one position to make it align.

Dealing with $\zeta_1 = 0$. A direct issue arises when $\zeta_1 = 0$: then $\zeta_1 - 1$ is invalid. We observe this happens only when the MSB of x is 1. Thus, we compute $d = \text{sign}(x)$ and shift by $\zeta_1 - 1 + d$. When $d = 1$, alignment is unnecessary, and we have $\zeta_1 - 1 + d = 0$. When $d = 0$, it performs the original protocol to shift $\zeta_1 - 1$.

For the protocol Π_{sign} , we directly adopt the scheme proposed by Lu et al. [32], which requires only two rounds of online communication with a total cost of $2\ell \log \ell$ bits. We found that their protocol can be modified to accept a secret share on ring $\mathbb{Z}_M$ and output the secret share of sign bit on an arbitrary-sized ring $\mathbb{Z}_K$, namely, $\langle z \rangle^K \leftarrow \Pi_{\text{sign}}^{M,K}(\langle x \rangle^M)$ (For details, refer to Appendix A.1.). For Π_{wrap1} , if the sign bit is fixed to 0, we can calculate it by $m_{x,(0)} \vee r_{x,(0)}$. If the sign bit is uncertain, we append a zero to the most significant bit of m_x and r_x , obtaining $\langle x' \rangle^{2^{\ell+1}}$ and then execute Π_{sign} on the zero-extended $\langle x' \rangle^{2^{\ell+1}}$ to obtain the wrap bit. (For specific protocol descriptions, refer to Appendix. A.2) In what follows, we give the construction of the other building blocks mentioned before.

4.1 The first non-zero element detection Π_{detect} .

We design a protocol $[\zeta]^K \leftarrow \Pi_{\text{detect}}([l_0]^p, \dots, [l_{\ell-1}]^p)$ to detect the position of the first non-zero element, namely, $l_\zeta = 1$ for the smallest ζ . Here, the ring $\mathbb{Z}_K$ can be arbitrary size and p should be prime. Fig. 5 illustrates our protocol Π_{detect} .

List Transformation. We first introduce the list transformation of Lu *et al.* [29] and [32] which converts a binary list into a new list in which only the first non-zero element is mapped to zero, while all other

Protocol $[\mathcal{D}]^K \leftarrow \Pi_{\text{prefix}}^{p,K}([l_0]^p, \dots, [l_{\ell-1}]^p), \langle b \rangle^K$

Input : Shared list $[\mathcal{L}] := ([l_0], \dots, [l_{\ell-1}])$.

Output : Shared list $[\mathcal{D}] := ([d_0], \dots, [d_{\ell-1}])$, while $d_i = 1$ for $i \leq \zeta$ and $d_i = 0$ for $i > \zeta$. ζ is the position of first non-zero element of $\mathcal{L}$.

Execution:

- P_i , for $i \in \{1, 2\}$, locally calculates $[l'_i] = (\sum_{t=0}^i [l_t]) - 2 \cdot [l_i] + 1 \bmod p$.
- P_i , for $i \in \{1, 2\}$, picks same random value $w_i \in \mathbb{Z}_p^*$ and random permutation $\pi : \mathbb{Z}_\ell \mapsto \mathbb{Z}_\ell$ with same seed;
- P_i , for $i \in \{1, 2\}$, calculates $([l''_0], \dots, [l''_{\ell-1}]) = \pi(w_0 \cdot [l'_0], \dots, w_{\ell-1} \cdot [l'_{\ell-1}])$;
- P_i , for $i \in \{1, 2\}$, reveals $(l''_0, \dots, l''_{\ell-1})$ to P_0 ;
- P_0 finds out ζ' where $l''_{\zeta'} = 0$;
- P_0 constructs list $s := (s_0, \dots, s_{\ell-1})$ where $s_{\zeta'} = 1$ and $s_i = 0$ for $i \neq \zeta'$;
- P_0 calculates list $\hat{s} := (s_0 \cdot r_b, \dots, s_{\ell-1} \cdot r_b)$;
- All parties invoke $[s]^K \leftarrow \Pi_{[\cdot]}(s)$ and $[\hat{s}]^K \leftarrow \Pi_{[\cdot]}(\hat{s})$;
- P_1 and P_2 set $\mathcal{D}^{(i)} = (d_j^{(i)})_{j \in \mathbb{Z}_\ell}$, where $d_j^{(i)} = 1$ for $j \leq \pi^-(i)$ and $d_j^{(i)} = 0$ for $j > \pi^-(i)$;
- P_1 and P_2 calculates $[d_j]^K = \sum_{i=0}^{\ell-1} d_j^{(i)} \cdot (m_b \cdot [s_i]^K + [\hat{s}_i]^K)$ for $j \in \mathbb{Z}_\ell$.

Figure 6: Prefix List Computation Protocol Π_{prefix} .

elements are mapped to a positive value. Let $\mathcal{L} := \{l_i\}_{i \in \mathbb{Z}_\ell}$ be the binary vector. The transformation is defined as.

$$\mathcal{L}' = \phi(\mathcal{L}) := \left\{ \left(\sum_{t=0}^i l_t \right) - 2 \cdot l_i + 1 \bmod p \right\}_{i \in \mathbb{Z}_\ell}.$$

That is, for each index i , the transformation first computes the prefix sum $l'_i = \sum_{t=0}^i l_t$, and the outputs $l''_i = l'_i - 2l_i + 1$. This transformation satisfies the following properties:

- If $l_i = 0$ and $l'_i = 0$, then $l''_i = 1$.
- If $l_i = 1$ and $l'_i = 1$, then $l''_i = 0$ (identifies the first non-zero bit).
- If $l_i = 0$ and $l'_i \geq 1$, then $l''_i = l'_i + 1 \geq 2$.
- If $l_i = 1$ and $l'_i \geq 2$, then $l''_i = l'_i - 1 \geq 1$.

Hence, the resulting list $\mathcal{L}'$ contains a unique zero at index ζ , corresponding to the first non-zero bit of $\mathcal{L}$, while all other entries are strictly positive. Since the evaluation of ϕ is entirely linear, P_1 and P_2 can locally apply ϕ to $([l_0], \dots, [l_{\ell-1}])$ to obtain $[\cdot]$ -secret shares of $\mathcal{L}'$. Note that all computations in ϕ are performed in $\mathbb{Z}_p$ where $p > \ell + 1$ to circumvent unintended wrap-around.

We aim to reveal $\mathcal{L}'$ to P_0 for the first non-zero element detection. For privacy, we let P_1 and P_2 randomize the new lists $\mathcal{L}'$ to ensure that opening them to P_0 leaks no information. First, to hide the zero's position, P_1 and P_2 use an agreed seed to derive identical random permutations π , and apply them to the lists $[\mathcal{L}']$. Then, to hide magnitude information, P_1 and P_2 sample identical random scalars $w_i \in \mathbb{Z}_p^*$ and perform $w_i \cdot l'_i$ for each term $i \in \mathbb{Z}_\ell$, so that zero remains zero and every nonzero entry remains nonzero under scaling. Finally, P_1 and P_2 open the permuted-and-scaled lists $\mathcal{L}''$ to P_0 . It holds that $l''_{\pi(i)} = w_i \cdot l'_i \pmod{p}$ for all $i \in \mathbb{Z}_\ell$, where $\mathcal{L}'' = \{l''_i\}_{i \in \mathbb{Z}_\ell}$.

So far, P_0 can detect the zero term index $\zeta' \in \mathbb{Z}_\ell$ for $l''_{\zeta'} = 0$. P_1 and P_2 hold π^- . We have $\pi^-(\zeta') = \zeta$. P_0 , according to ζ' , generates a one-hot list s where only $s_{\zeta'} = 1$. Then, P_0 secret-shares s

Protocol $\langle 2^\zeta \rangle^{2^\ell} \leftarrow \Pi_{\text{pow2}}^\ell([\zeta]^{2^\ell})$

Input : $[\cdot]^{2^\ell}$ -shared value of ζ .

Output : $\langle \cdot \rangle^\ell$ -shared value of 2^ζ .

Execution:

- All parties invoke $[r_z]^{2^\ell} \leftarrow \mathcal{F}_{\text{share}}[2^\ell]$;
- Each party P_i , for $i \in \{1, 2\}$, locally calculates $d'_i = 2^{\lceil \zeta \rceil_i} \pmod{2^{3\ell}}$.
- P_1 and P_2 invoke $[d'] \leftarrow \Pi_{\text{mult}}^{2^\ell}(d'_1, d'_2)$;
- P_1 and P_2 invoke $[\gamma]^{2^\ell} \leftarrow \Pi_{\text{mult}}^{2^\ell}([\zeta]_{1,(0)}^{2^\ell}, [\zeta]_{2,(0)}^{2^\ell})$ and set $[c] = [\zeta]_{1,(0)}^{2^\ell} + [\zeta]_{2,(0)}^{2^\ell} - [\gamma]$;
- P_1 and P_2 invoke $[\gamma']^{2^\ell} \leftarrow \Pi_{\text{mult}}^{2^\ell}([d']_{1,(\ell)}, [d']_{2,(\ell)})$ and set $[\beta] = [d']_{1,(\ell)} + [d']_{2,(\ell)} - [\gamma']$;
- P_i , for $i \in \{1, 2\}$, locally calculate
 - $[a^{(0)}]_i = [d']_i \pmod{2^\ell}$;
 - $[a^{(1)}]_i = \lfloor [d']_i / 2^{2\ell} \rfloor + [\beta]_i \pmod{2^\ell}$;
- All parties invoke $[\gamma'']^{2^\ell} \leftarrow \Pi_{\text{mult}}^{2^\ell}([c], [a^{(1)}] - [a^{(0)}])$ and set $[2^\zeta] = [\gamma''] + [a^{(0)}]$;
- P_1 and P_2 reveal $[m_z] = [2^\zeta] - [r_z]$ to each other;
- P_i , for $i \in \{1, 2\}$, outputs $(m_z, [r_z]_i)$, P_0 outputs $([r_z]_1, [r_z]_2)$.

Figure 7: Power-of-Two Protocol Π_{pow2} .

with P_1 and P_2 . Based on the shares, P_1 and P_2 locally compute

$$[\zeta] = \sum_{i=0}^{\ell-1} \pi^-(i) \cdot [s_i].$$

Notice that the secret share $[\zeta]$ can reside over any ring or field, depending on the algebraic structure in which P_0 secret-shares s_i .

Efficiency. Our first non-zero element detection protocol $\Pi_{\text{detect}}^{p,K}$ costs 2 round with communication of $\ell(2 \log p + \log K)$ bits.

4.2 The prefix list computation protocol Π_{prefix} .

Based on Π_{detect} , we further develop the prefix list computation protocol $[D]^K \leftarrow \Pi_{\text{prefix}}^{p,K}([\mathcal{L}]^p, \langle b \rangle)$, where all entries in the prefix list D up to the first non-zero element of $\mathcal{L}$ are set to b , and the remaining entries are set to 0. As before, the protocol supports the case where the input and output reside in different rings ($\mathbb{Z}_p$ and $\mathbb{Z}_K$).

A straightforward approach would be to first invoke Π_{detect} to obtain the index ζ of the first non-zero element in $\mathcal{L}$, and then generate the corresponding list $\mathcal{D}$ from ζ . However, by revisiting the protocol flow of Π_{detect} , we observe that once P_0 learns the permuted index ζ' and parties P_1 and P_2 hold the permutation π , the prefix list $\mathcal{D}$ can be computed more directly.

Concretely, we let P_1 and P_2 locally construct, for each inverse permutation $\pi^-(i)$, a prefix list

$$\mathcal{D}^{(i)} = (\underbrace{1, 1, \dots, 1}_{\pi^-(i)}, \underbrace{0, 0, \dots, 0}_{\ell - \pi^-(i)})$$

After P_0 secret-shares the one-hot list s corresponding to ζ' , P_1 and P_2 can directly select the correct

Protocol $(\langle z \rangle^K) \leftarrow \Pi_{\text{convert}}^{M,K}([x]^M)$

P_i and P_j hold the same seed $\eta^{i,j}$.

Input : $[\cdot]^M$ -shared value of x , the first bit of x is 0.

Output : $\langle \cdot \rangle^K$ -shared value of z , while $z = x$.

Execution:

- P_i , for $i \in \{1, 2\}$, and P_0 generate same share $[r_z]_i^K$ using seed $\eta^{0,i}$;
- P_i , for $i \in \{1, 2\}$ invoke $[c]^K \leftarrow \Pi_{\text{mult}}^K([x]_{1,(0)}^M, [x]_{2,(0)}^M)$
- P_1 calculates $[m_z]_1^K = [x]_1^M - M \cdot ([x]_{1,(0)}^M - [c]_1^K) - [r_z]_1^K \pmod{K}$;
- P_2 calculates $[m_z]_2^K = [x]_2^M - M \cdot ([x]_{2,(0)}^M - [c]_2^K) - [r_z]_2^K \pmod{K}$;
- P_1 and P_2 reveal m_z to each other.

Figure 8: Secure Conversion Protocol.

prefix list $D^{(i)}$ by computing

$$[\mathcal{D}] = \sum_{i=0}^{\ell-1} \mathcal{D}^{(i)} \cdot [s_i \cdot b].$$

Here, by multiplying s_i with b , we ensure that the output is an all-zero list when $b = 0$, and $\mathcal{D}^{(i)} \cdot [s_i \cdot b]$ denotes that each element in $\mathcal{D}^{(i)}$ is multiplied by $[s_i \cdot b]$. In addition, we observe that by setting $[s'_i] = [s_i \cdot b]$, it can be computed from s_i and $\langle b \rangle$ within one round. Noting that $s'_i = s_i \cdot (m_b + r_b)$, P_0 can secret-share both s_i and $\hat{s}_i = s_i \cdot r_b$, after which P_1 and P_2 locally evaluate $[s'_i] = m_b \cdot [s_i] + [\hat{s}_i]$. Accordingly, we replace the part in the original protocol Π_{detect} that selects $[\zeta]$ via π^- with selecting the prefix list $\mathcal{D}^{(i)}$ through π^- . This yields the desired prefix list $\mathcal{D}$ associated with ζ directly. Fig. 6 illustrates the resulting protocol Π_{prefix} .

Efficiency. Our prefix list computation protocol $\Pi_{\text{prefix}}^{p,K}$ costs 2 round with communication of $2\ell(\log p + \log K)$ bits.

4.3 The left-shift protocol Π_{shift} .

We design a constant-round left-shift protocol $\langle x' \rangle \leftarrow \Pi_{\text{shift}}(\langle x \rangle, [\zeta])$ which left-shift x by ζ position. We first observe that a left-shift by ζ bits is equivalent to multiplication by 2^ζ . Accordingly, to left-shift $\langle x \rangle$ by ζ , we first compute $\langle 2^\zeta \rangle$ from $[\zeta]$, and then invoke the secure multiplication protocol to obtain $x' = x \cdot 2^\zeta$.

A key observation is the following. Let $[\zeta]_1 + [\zeta]_2 = \zeta + c\ell < 2\ell$, where $[\zeta]$ is a secret share over $\mathbb{Z}_\ell$ and c indicates whether $[\zeta]_1 + [\zeta]_2$ wraps around modulo ℓ . Then we have $2^{[\zeta]_1} \cdot 2^{[\zeta]_2} = 2^{\zeta+c\ell}$. By secret-sharing $2^{[\zeta]_1}$ and $2^{[\zeta]_2}$ over $\mathbb{Z}_{2\ell}$, and computing $[d]^{2\ell} = [2^{[\zeta]_1}]^{2^{2\ell}} \cdot [2^{[\zeta]_2}]^{2^{2\ell}}$, we obtain a value d with the following structure: if $c = 0$, then 2^ζ is stored in the lower ℓ bits of d ; if $c = 1$, then 2^ζ is stored in the upper ℓ bits of d . We can therefore split d into its high and low ℓ -bit parts, and use c to select the correct one to recover 2^ζ . The main technical challenges addressed by our protocol are thus (i) how to securely compute the wrap bit c , and (ii) how to efficiently decompose d into its high and low ℓ -bit parts.

Calculation of c . To calculate the wrap bit c from $[\zeta]$ easily, as mentioned in Section 3, we can make the most significant bit of ζ be 0 and apply the observation of mentioned in Section 3 to efficiently compute the wrap bit as $c = [\zeta]_{1,(0)} \wedge [\zeta]_{2,(0)}$. To ensure the most significant bit of ζ be 0, we let ζ shared over $\mathbb{Z}_{2\ell}$ (Since $d = 2^{[\zeta]_1} \cdot 2^{[\zeta]_2} = 2^{\zeta+2c\ell} < 2^{3\ell}$ over $\mathbb{Z}_{2^{3\ell}}$, the multiplication $2^{[\zeta]_1} \cdot 2^{[\zeta]_2}$ is carried out over the ring $\mathbb{Z}_{2^{3\ell}}$).

Splitting of d . For the case where 2^ζ resides in the lower bits, the secret share $[d]$ discards the higher bits, and each party can perform a local modulo 2^ℓ operation to obtain the result. That is, $[2^\zeta]_j = [d]_j^{2^{3\ell}} \pmod{2^\ell}$. However, when 2^ζ lies in the higher bits, directly keeping the upper ℓ bits may introduce a

carry error, since one carry bit from the lower part may be dropped. In this case, we need to check for the carry from the lower bits, namely the wrap bit, after discarding the upper ℓ bits. Because the lower part is zero in this case, we can again compute this carry as $[\beta] = [d]_{1,(\ell)}^{2^{3\ell}} \vee [d]_{2,(\ell)}^{2^{3\ell}}$ ($[d]_{1,(\ell)}^{2^{3\ell}}$ represents the ℓ^{th} bit of $[d]_1^{2^{3\ell}}$). Accordingly, we obtain the two decompositions of $[d]$ as follows:

$$\begin{aligned} [d_1]^{2^\ell} &= ([d]_1^{2^{3\ell}} \bmod 2^\ell, [d]_2^{2^{3\ell}} \bmod 2^\ell) \\ [d_2]^{2^\ell} &= \left(\left\lfloor \frac{[d]_1^{2^{3\ell}}}{2^{2\ell}} \right\rfloor + [\beta]_1 \bmod 2^\ell, \left\lfloor \frac{[d]_2^{2^{3\ell}}}{2^{2\ell}} \right\rfloor + [\beta]_2 \bmod 2^\ell \right). \end{aligned}$$

Fig. 7 illustrates the procedure for converting $[\zeta]$ into $\langle 2^\zeta \rangle$. Finally, the left-shift protocol $\Pi_{\text{shift}}(\langle x \rangle^{2^\ell}, [\zeta]^{2^\ell})$ invokes the secure multiplication protocol Π_{mult} to compute $\langle x' \rangle = \langle 2^\zeta \cdot x \rangle$.

Efficiency. Our left-shift protocol Π_{shift} costs 5 round with communication of 14ℓ bits in the online phase and 6ℓ bits in the offline phase.

4.4 The alignment protocol Π_{align} .

In this section, we present the complete construction of the alignment protocol Π_{align} by integrating the aforementioned sub-protocols.

Fig. 9 depicts the procedure of our alignment protocol. We first need to calculate $\mathcal{L}_1 = (m_{x,(i)} \vee r_{x,(i)})_{i \in \mathbb{Z}_\ell}$ and $\mathcal{L}_2 = (m_{x,(i)} \wedge r_{x,(i)})_{i \in \mathbb{Z}_\ell}$ for each bit i . For $m_{x,(i)} \wedge r_{x,(i)}$, it can be calculated as $m_{x,(i)} \cdot r_{x,(i)}$. For $m_{x,(i)} \vee r_{x,(i)}$, we have $m_x[i] \vee r_{x,(i)} = m_{x,(i)} + r_{x,(i)} - m_{x,(i)} \cdot r_{x,(i)}$. As noted above, we need to compute $\mathcal{L}$ over $\mathbb{Z}_p$. We let P_0 in the offline phase secret-share $r_{x,(i)}$ over $\mathbb{Z}_p$ for all $i \in \mathbb{Z}_\ell$. In steps 1-2, P_1 and P_2 obtain $[r_{x,(i)}]$, after which they can locally compute $m_{x,(i)} \cdot [r_{x,(i)}]$ and $m_{x,(i)} + [r_{x,(i)}] - m_{x,(i)} \cdot [r_{x,(i)}]$ to obtain the $[\cdot]$ -secret-shared lists $\mathcal{L}_1$ and $\mathcal{L}_2$. Then, in protocol steps 3-5, we use Π_{wrap1} to obtain the wrap bit b , and subsequently use the wrap bit together with $\mathcal{L}_2$ to compute the prefix list $\mathcal{D}$, where $\mathcal{D}$ becomes an all-zero list when $b = 0$. Meanwhile, we compute $\text{sign}(x)$ for further avoiding from -1 in $\zeta - 1$. Notably, as described in Appendix A.3, we implement a protocol that computes the wrap bit and sign bit simultaneously, which is much faster than computing them separately. We can use $\Pi_{\text{wrap2/sign}}^{2^\ell, 2\ell, p}$ to replace these two steps and reduce the cost. In step 6, we use $\mathcal{D}$ to remove the leading 1s in $\mathcal{L}_1$. In step 7, we compute $[\zeta_1]$ from the updated list, where the first non-zero bit corresponds to ζ_1 or $\zeta_1 - 1$. In step 8, we left-shift x by $\zeta_1 - 1 + d$ positions, and in steps 9-12, we check whether the first bit c after the shift equals 1 to decide whether to further left-shift by one more position to reach ζ_1 . It can be observed that when $c = 1$, the first non-zero bit is at position $\zeta_1 - 1 + d$; when $c = 0$, it is at position ζ_1 . Therefore, in steps 13-14, we set the final position of the first non-zero bit as $\zeta = \zeta_1 - c + d$. Finally, Π_{convert} is used to convert the $[\cdot]$ -type secret sharing into $\langle \cdot \rangle$ -type secret sharing, and its procedure is illustrated in Fig. 8.

Efficiency. Our alignment protocol $\Pi_{\text{align}}^{2^\ell, 2\ell, p}$ costs 12 round with communication of $10\ell \log \ell + 3\ell \log 2\ell + 18\ell + 2 \log \ell + 4\ell_e$ bits in the online phase and requires communication of $4\ell \log \ell + 7\ell + \ell_e$ bits in the offline phase.

5 Floating-point Arithmetic Framework.

In this section, we present our floating-point arithmetic framework. Floating-point computation can be regarded as performing fixed-point computation on the mantissa bits followed by an alignment operation. We implement floating-point computation by applying the fixed-point multiplication protocol from Section 3 and the alignment protocol from Section 4.

Floating-point Structure under Share. As we defined earlier, a floating-point number $\bar{x} := (b_x, \hat{x}, \tilde{x})$ consists of a sign bit b_x , a fixed-point mantissa $\hat{x} \in [2^{\ell_f}, 2^{\ell_f+1})$, and an exponent $\tilde{x} \in [0, 2^{\ell_e} - 1]$. The corresponding real value is given by $x = (-1)^{b_x} \cdot \hat{x} \cdot 2^{\tilde{x} - 2^{\ell_e} - 1}$. While mantissa multiplication can be performed within the original ring size using the multiplication protocol from Section 3, floating-point

Protocol $(\langle z \rangle^{2^\ell}, \langle \zeta \rangle^{2^{\ell_e}}) \leftarrow \Pi_{\text{align}}^{2^\ell, 2^{\ell_e}}(\langle x \rangle^{2^\ell})$

Input : $\langle \cdot \rangle^{2^\ell}$ -shared value of x .

Output : $\langle \cdot \rangle^{2^{\ell_e}}$ -shared value of $\zeta = \text{fnz}(x)$ and $\langle \cdot \rangle^{2^\ell}$ -shared value of $z = x \cdot 2^\zeta$, $\ell_e \leq \ell$.

Preprocessing:

- P_0 extracts r_x as $\{r_{x,0}, \dots, r_{x,\ell-1}\}$ and performs $[r_{x,i}]^p \leftarrow \Pi_{[\cdot]}^p(r_{x,i})$ for $i \in \mathbb{Z}_\ell$, taking the biggest prime of $p \in (\ell, 2^{\log \ell + 1}]$;

Online:

1. P_1 and P_2 bit-extract m_x as $\{m_{x,i} \in \{0, 1\}\}_{i \in \mathbb{Z}_\ell}$ while $\sum_{i=0}^{\ell-1} 2^{\ell-2-i} m_{x,i} = m_x$;
2. P_1 and P_2 calculate $[\mathcal{L}_2]^p := (m_{x,i} \cdot [r_{x,i}]^p)_{i \in \mathbb{Z}_\ell}$; and $[\mathcal{L}_1]^p := (m_{x,i} + [r_{x,i}]^p - m_{x,i} \cdot [r_{x,i}]^p)_{i \in \mathbb{Z}_\ell}$;
3. All parties invoke $\langle d \rangle^{2^\ell} \leftarrow \Pi_{\text{sign}}^{2^\ell, 2^\ell}(\langle x \rangle^{2^\ell})$;
4. All parties invoke $\langle b \rangle^p \leftarrow \Pi_{\text{wrap1}}^{2^\ell, p}(\langle x \rangle^{2^\ell})$;
5. All parties invoke $[\mathcal{D}]^p \leftarrow \Pi_{\text{prefix}}^{p,p}([\mathcal{L}_2]^p, \langle b \rangle^p)$;
6. P_1 and P_2 set $[\mathcal{L}_1]^p = [\mathcal{L}_1]^p - [\mathcal{D}]^p$;
7. All parties perform $[\zeta_1]^{2^\ell} \leftarrow \Pi_{\text{detect}}^{p, 2^\ell}([\mathcal{L}_1]^p)$;
8. All parties perform $\langle x' \rangle^{2^\ell} \leftarrow \Pi_{\text{shift}}^{2^\ell}(\langle x \rangle^{2^\ell}, [\zeta_1]^{2^\ell} - 1 + [d]^{2^\ell})$;
9. All parties set $\langle x'' \rangle^{2^\ell} = \langle x' \rangle^{2^\ell} \cdot 2$;
10. All parties perform $\langle c \rangle^{2^\ell} \leftarrow \Pi_{\text{sign}}^{2^\ell, 2^\ell}(\langle x' \rangle^{2^\ell})$;
11. All parties invoke $\Pi_{\text{mult}}^{2^\ell}$ to evaluate $\langle z \rangle^{2^\ell} = \langle c \rangle^{2^\ell} (\langle x' \rangle^{2^\ell} - \langle x'' \rangle^{2^\ell}) + \langle x'' \rangle^{2^\ell}$.
12. All parties discard the higher-order $\ell - \ell_e$ bits of $\langle c \rangle^{2^\ell}$ to obtain $\langle c \rangle^{2^{\ell_e}}$.
13. All parties invoke $\langle \zeta_1 \rangle^{2^{\ell_e}} \leftarrow \Pi_{\text{convert}}^{2^\ell, 2^{\ell_e}}([\zeta_1]^{2^\ell} + [d]^{2^\ell})$;
14. All parties output $(\langle z \rangle^{2^\ell}, \langle \zeta_1 \rangle^{2^{\ell_e}} - \langle c \rangle^{2^{\ell_e}})$

Figure 9: Secure Alignment Protocol.

Protocol ($\llbracket z \rrbracket$) $\leftarrow \Pi_{\text{float-mult}}(\llbracket \bar{x} \rrbracket, \llbracket \bar{y} \rrbracket)$

Input : $\llbracket \cdot \rrbracket$ -shared value of $\bar{x}$ and $\bar{y}$.

Output : $\llbracket \cdot \rrbracket$ -shared value of $\bar{z}$, while $z = x \cdot y$.

Execution:

- All parties perform:
 - Locally calculate $\langle b_z \rangle^2 = \langle b_x \rangle^2 \oplus \langle b_y \rangle^2$;
 - Invoke $\langle \hat{z} \rangle^{3+\ell_f} \leftarrow \Pi_{\text{fix-mult}}^{2, \ell_f, \ell_f+3}(\langle \hat{x} \rangle, \langle \hat{y} \rangle)$;
 - Drop the first bit of $\langle \hat{z} \rangle^{3+\ell_f}$ to obtain $\langle \hat{z}' \rangle^{2+\ell_f}$.
 - Invoke $\langle c \rangle^{2^{3+\ell_f}} \leftarrow \Pi_{\text{sign}}^{2^{2+\ell_f}, 2^{3+\ell_f}}(\langle \hat{z}' \rangle^{2^{2+\ell_f}})$;
 - Invoke $\langle \hat{z} \rangle^{2^{3+\ell_f}} \leftarrow \Pi_{\text{mult}}^{2^{3+\ell_f}}(\langle \hat{z} \rangle^{2^{3+\ell_f}}, 2 - \langle c \rangle^{2^{3+\ell_f}})$;
 - Drop the lowest bit of $\langle \hat{z} \rangle^{2^{3+\ell_f}}$ to obtain $\langle \hat{z} \rangle^{2^{2+\ell_f}}$;
 - Set $\langle c \rangle^{2^{\ell_e}} = \langle c \rangle^{2^{3+\ell_f}} \pmod{2^{\ell_e}}$;
 - Set $\langle \tilde{z} \rangle^{2^{\ell_e}} = \langle \tilde{x} \rangle^{2^{\ell_e}} + \langle \tilde{y} \rangle^{2^{\ell_e}} + \langle c \rangle^{2^{\ell_e}} - 2^{\ell_e-1}$;
 - Output $\llbracket \bar{z} \rrbracket := (\langle b_z \rangle^2, \langle \hat{z} \rangle^{2^{\ell_f+2}}, \langle \tilde{z} \rangle^{2^{\ell_e}})$.

Figure 10: Secure Floating Point Multiplication Protocol.

addition requires special consideration. For mantissas in the range $[1, 2)$, after alignment and addition, the result falls within $[2, 4)$. This necessitates an additional bit of storage to prevent overflow. Therefore, for a mantissa of $\ell_f + 1$ bits, we secret-share it over a $\ell_f + 2$ -bit ring to avoid overflow during addition. For the sign bit b_x , we directly secret-share it over a 1-bit ring, and for the exponent of ℓ_e bits, we secret-share it over an ℓ_e -bit ring. Consequently, under secret sharing, our floating-point number is represented as

$$\llbracket \bar{x} \rrbracket := (\langle b_x \rangle^2, \langle \hat{x} \rangle^{2^{\ell_f+2}}, \langle \tilde{x} \rangle^{2^{\ell_e}}).$$

It is worth noting that, due to space limitations, we do not discuss the handling of zero-valued floating-point numbers. To address cases where the floating-point value is zero, one can adopt the method proposed by Rathee *et al.* [40], which introduces an additional zero bit and performs a zero-value check after computation, while using the zero bit for result selection after computation. Meanwhile, in this work, we only design and implement floating-point addition and multiplication. Other operations, such as division, can be realized based on addition and multiplication using techniques like polynomial approximation or the Newton–Raphson iteration method.

5.1 Multiplication.

For floating-point multiplication, we directly perform multiplication on the mantissas, addition on the exponent parts (need to subtract the bias 2^{ℓ_e-1}), and XOR on the sign bits. For the mantissas $\langle \hat{x} \rangle$ and $\langle \hat{y} \rangle$, we can directly apply $\Pi_{\text{mult}}^{2, \ell_f}$ to calculate $\langle \hat{z} \rangle = \langle \hat{x} \cdot \hat{y} / 2^{\ell_f} \rangle$.

To handle the possible carry that may occur after mantissa multiplication, which could cause the most significant bit to become 1, an alignment operation is required. Specifically, under protocol $\Pi_{\text{mult}}^{2, \ell_f}$, there are only two possible cases: i) The product remains within the normalized range, namely, $\langle \hat{z} \rangle \in [2^{\ell_f}, 2^{\ell_f+1})$, indicating no carry occurs. ii) The product of two normalized mantissas exceeds 2^{ℓ_f+1} resulting in a one-bit carry. In this case, a right-shift operation is performed on the mantissa $\langle \hat{z} \rangle$ to restore it to the normalized form, and the exponent is incremented by 1 accordingly. We use the protocol $\langle c \rangle \leftarrow \Pi_{\text{sign}}(\langle \hat{z} \rangle)$ to determine whether the most significant bit (MSB) of the multiplication result is 1. To simplify computation, for the result c , we first perform the operation $\hat{z} \cdot (2 - c)$, which aligns the MSB to 1. The correctness can be straightforwardly verified: when the MSB is 1, $\hat{z}$ remains unchanged;

Protocol ($\llbracket z \rrbracket$) $\leftarrow$ $\Pi_{\text{float-add}}(\llbracket \bar{x} \rrbracket, \llbracket \bar{y} \rrbracket)$

Input : $\llbracket \cdot \rrbracket$ -shared value of $\bar{x}$ and $\bar{y}$.

Output : $\llbracket \cdot \rrbracket$ -shared value of $\bar{z}$, while $z = x + y$.

Execution:

- Invoke $\langle c_1 \rangle^2 \leftarrow \Pi_{\text{great}}(\langle \tilde{x} \rangle, \langle \tilde{y} \rangle)$, $\langle c_2 \rangle^2 \leftarrow \Pi_{\text{eq}}(\langle \tilde{x} \rangle, \langle \tilde{y} \rangle)$ and $\langle c_3 \rangle^2 \leftarrow \Pi_{\text{great}}(\langle \hat{x} \rangle, \langle \hat{y} \rangle)$;
- Invoke Π_{mult}^2 to evaluate $\langle c \rangle^2 = \langle c_2 \rangle^2 \cdot \langle c_3 \rangle^2 + (1 - \langle c_2 \rangle^2) \langle c_1 \rangle^2$;
- Perform $(\llbracket \chi \rrbracket, \llbracket \gamma \rrbracket) \leftarrow \Pi_{\text{select}}(\langle c \rangle^2, (\llbracket \bar{x} \rrbracket, \llbracket \bar{y} \rrbracket), (\llbracket \bar{y} \rrbracket, \llbracket \bar{x} \rrbracket))$;
- Set $\langle \zeta_1 \rangle^{2^{\ell_e}} = \langle \tilde{\chi} \rangle^{2^{\ell_e}} - \langle \tilde{\gamma} \rangle^{2^{\ell_e}}$;
- Perform $\langle d \rangle^2 \leftarrow \Pi_{\text{great}}(\langle \zeta_1 \rangle, \ell_f)$;
- Perform $\langle e \rangle^{2^{2+\ell_f}} \leftarrow \Pi_{\text{wrap2}}^{2^{2+\ell_f}, 2^{2+\ell_f}}(\langle \hat{\gamma} \rangle)$;
- Set $\langle \hat{\gamma} \rangle^{2^{2+\ell_f}} := \langle \hat{\gamma} \rangle^{2^{2+\ell_f}} - 2^{\ell_f+2} \cdot \langle e \rangle \pmod{2^{2+2\ell_f}}$;
- Perform left-shift protocol $\langle \hat{\gamma}' \rangle \leftarrow \Pi_{\text{shift}}(\langle \hat{\gamma} \rangle^{2^{2+\ell_f}}, \ell_f - \lceil \zeta_1 \rceil)$;
- Drop the lowest ℓ_f of $\langle \hat{\gamma}' \rangle^{2^{2+\ell_f}}$ to obtain $\langle \hat{\gamma}' \rangle^{2^{2+\ell_f}}$;
- Perform $\langle \hat{\gamma}' \rangle \leftarrow \Pi_{\text{select}}(\langle d \rangle, \langle 0 \rangle, \langle \hat{\gamma}' \rangle)$;
- Calculate $\langle \hat{z}_1 \rangle^{2^{2+\ell_f}} = \langle \hat{\chi} \rangle^{2^{2+\ell_f}} + \langle \hat{\gamma}' \rangle^{2^{2+\ell_f}}$ and $\langle \hat{z}_2 \rangle^{2^{2+\ell_f}} = \langle \hat{\chi} \rangle^{2^{2+\ell_f}} - \langle \hat{\gamma}' \rangle^{2^{2+\ell_f}}$;
- Calculate $\langle g \rangle^2 = \langle b_\chi \rangle^2 + \langle b_\gamma \rangle^2 \pmod{2}$ and set $\langle b_z \rangle^2 = \langle b_\chi \rangle^2$;
- Perform $\langle \hat{z} \rangle^{2^{2+\ell_f}} \leftarrow \Pi_{\text{select}}(\langle g \rangle^2, \langle \hat{z}_2 \rangle^{2^{2+\ell_f}}, \langle \hat{z}_1 \rangle^{2^{2+\ell_f}})$;
- Perform $(\langle \hat{z} \rangle^{2^{2+\ell_f}}, \langle \zeta \rangle^{2^{\ell_e}}) \leftarrow \Pi_{\text{align}}^{2^{2+\ell_f}, 2^{\ell_e}}(\langle \hat{z} \rangle)$;
- Perform $\langle \hat{z} \rangle^{2^{2+\ell_f}} \leftarrow \Pi_{\text{trunc}}^{2^{2+\ell_f}, 1}(\langle \hat{z} \rangle^{2^{2+\ell_f}})$ and set $\langle \tilde{z} \rangle^{2^{\ell_e}} = \langle \tilde{\chi} \rangle^{2^{\ell_e}} - \langle \zeta \rangle^{2^{\ell_e}} + 1$;
- Output $\llbracket \bar{z} \rrbracket := (\langle b_z \rangle^2, \langle \hat{z} \rangle^{2^{2+\ell_f}}, \langle \tilde{z} \rangle^{2^{\ell_e}})$.

Figure 11: Secure Floating Point Addition Protocol.

Table 1: Test Results from Fixed Point Multiplication

(l_i, l_f)	Protocol	MAN setting			WAN setting		
		2^{16}	2^{20}	2^{24}	2^{16}	2^{20}	2^{24}
(8, 8)	SOTA(Online)	31.34	471.82	7392.53	85.13	1313.80	19027.91
	Ours(Online)	14.20	142.91	2132.11	32.86	359.34	5429.75
	SOTA(Total)	51.39	728.65	11199.67	127.65	2091.81	28644.09
	Ours(Total)	43.79	500.92	7010.34	87.47	1212.01	17663.24
(8, 16)	SOTA(Online)	39.34	613.80	9551.94	106.08	1634.84	24642.26
	Ours(Online)	15.79	227.69	3197.66	43.92	610.30	8182.64
	SOTA(Total)	63.24	949.99	14433.62	157.98	2573.99	36949.82
	Ours(Total)	62.32	849.96	12208.83	165.19	2136.80	31179.26
(16, 16)	SOTA(Online)	43.20	684.97	10564.98	116.22	1817.03	27060.53
	Ours(Online)	20.63	279.87	4262.78	54.35	691.98	10864.68
	SOTA(Total)	68.39	1045.27	15964.06	176.07	2808.05	40578.23
	Ours(Total)	70.32	930.65	13793.21	178.85	2309.92	35183.12

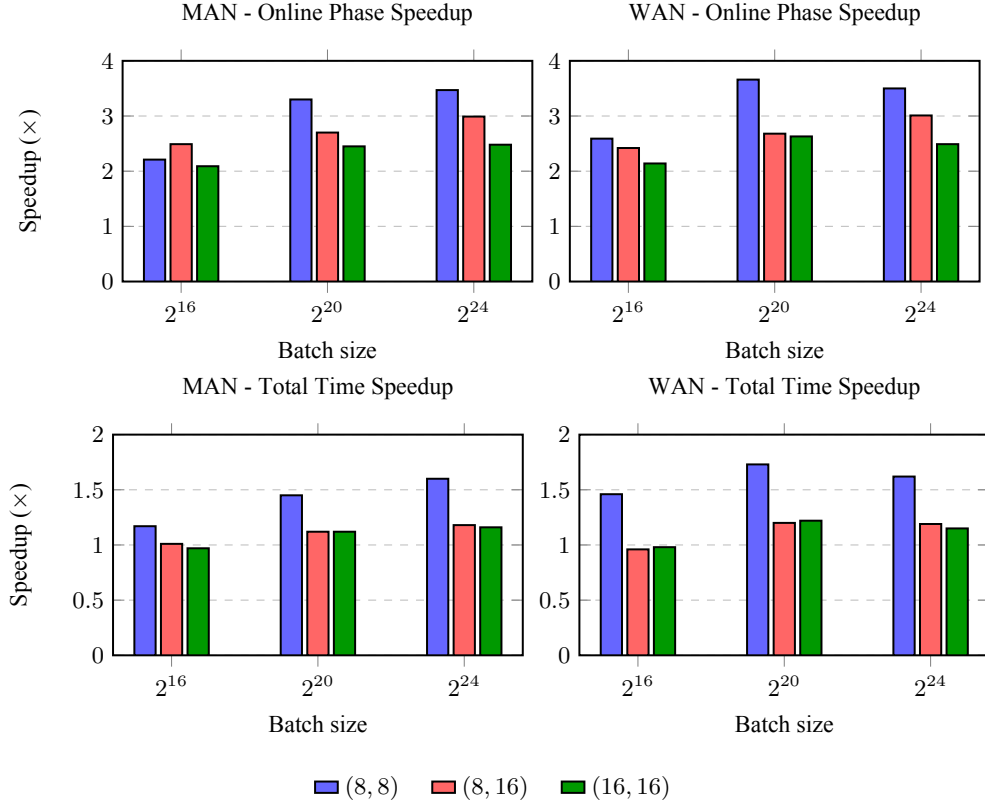


Figure 12: Speedup of our protocol over SOTA for fixed-point multiplication across different bit-length configurations (l_i, l_f) and batch sizes. Speedup is calculated as SOTA time / Our time. Top row shows online phase speedup (up to $3.7\times$), bottom row shows total execution time speedup (up to $1.7\times$). Left column: MAN setting, right column: WAN setting.

when the MSB is 0, $\hat{z}$ is left-shifted by one bit to set the MSB to 1. Then, we execute a right-shift-by-one operation to restore the normalized form.

In a conventional approach, the right shift is done by locally shifting the secret shares by one bit.

However, since the wrap bit may appear in the sign bit position after the shift, we must compute the wrap bit and remove its effect after shifting.

We observe that if we use the protocol $\Pi_{\text{fix-mult}}$ to compute $\hat{x} \cdot \hat{y}$ while outputting one additional high bit—resulting in $\hat{z}^{3+\ell_f}$ (where the most significant bit is 0)—then during the right-shift operation, we can simply discard the least significant bit to obtain $\hat{z}^{2+\ell_f}$, without the need for the wrap-bit computation step. By setting the input integer bit length to 2 and the fractional bit length to ℓ_f , and the output length to $3 + \ell_f$, we obtain the fixed-point multiplication protocol $\Pi_{\text{fix-mult}}^{2,\ell_f,3+\ell_f}$ as described above. In this case, since we now have an additional high bit, we need to compute the sign bit based on the second most significant bit. Thus, when calculating the sign bit, we simply discard the highest bit and perform Π_{sign} on the remain part. Fig. 10 depicts our floating point multiplication protocol.

5.2 Addition.

Floating-point addition is considerably more complex than multiplication. For floating-point addition, our overall approach follows that of Secfloat[40], and proceeds as follows. First, for the two values $\bar{x}$ and $\bar{y}$ to be added and result $\bar{z}$, we first use mantissas and exponents of $\bar{x}$ and $\bar{y}$ to sort $\bar{x}$ and $\bar{y}$, obtaining $\bar{\chi}$ and $\bar{\gamma}$, where the corresponding real-number value holds $|\chi| > |\gamma|$. For the mantissas $\hat{\chi}, \hat{\gamma}$ and exponents $\tilde{\chi}, \tilde{\gamma}$, we then right-shift $\hat{\gamma}$ by $\tilde{\chi} - \tilde{\gamma}$ so that $\bar{\gamma}$'s exponent matches that of $\bar{\chi}$, and perform the addition on the mantissas. When taking the sign bits (b_χ for $\bar{\chi}$ and b_γ for $\bar{\gamma}$) into account, the sign of the result follows that of $\bar{\chi}$; thus, we directly set $b_z = b_\chi$. For numbers with the same sign, we have $z = \chi + \gamma$; for opposite signs, $z = \chi - \gamma$. We first compute the XOR of the sign of χ and γ to obtain a control bit c , then perform both addition and subtraction on their mantissas, and finally use c to select the correct mantissa result.

Comparison. To compare x and y , we first compare their exponents and use the Π_{eq} protocol to test if they are equal. If the exponents are equal, we then compare the mantissas. Specifically, we employ Π_{great} to compute $c_1 = (\tilde{x} \stackrel{?}{>} \tilde{y})$, Π_{eq} to compute $c_2 = (\tilde{x} \stackrel{?}{=} \tilde{y})$, and $\langle x \rangle^K \leftarrow \Pi_{\text{great}}^{M,K}(\langle x \rangle^M, \langle y \rangle^M)$ to compute $c_3 = (\hat{x} \stackrel{?}{>} \hat{y})$. The final result is then given by $c := x \stackrel{?}{>} y = c_2 \cdot c_3 + (1 - c_2) \cdot c_1$. Then, we use c to select (χ, γ) from the pairs (y, x) and (x, y) . If $c = 1$, then $(\chi, \gamma) = (x, y)$; if $c = 0$, then $(\chi, \gamma) = (y, x)$. For the protocol Π_{great} , we adopt the 2-round scheme proposed by Lu *et al.* (refer to Appendix. A.4 for details). For Π_{eq} , we design a constant-round protocol, with specific content provided in Appendix. A.5. Regarding secure selection, we have implemented $\Pi_{\text{select}}(\langle b \rangle, \langle X \rangle, \langle Y \rangle)$, which can select and output one of the secret-shared lists X or Y based on the secret-shared bit b (refer to Appendix. A.6 for details).

So far, we obtain a not-yet-normalized $\bar{z} := (b_z, \hat{z}, \tilde{z} = \tilde{\chi})$. We perform the alignment operation to normalize it. It is worth noting that for operands χ and γ with the same sign, the addition can at most cause a one-bit carry. Since our floating-point mantissa encoding reserves one zero bit at the most significant position, overflow will not occur. For χ and γ with the different sign, subtraction may cause the first non-zero bit to appear at an arbitrary position. Considering that addition requires a right shift for alignment while subtraction requires a left shift for alignment, we employ Π_{align} to uniformly move the first non-zero bit to the highest position, followed by a right-shift-by-one operation.

The only remaining challenge here is that when we align γ , we need to perform a right shift on the mantissa of γ . In previous sections, we only implemented left shifts but not right shifts. We can implement right shifts through left shifts. Specifically, for an ℓ' -bit secret-shared value $\hat{\gamma}$, we first zero-pad it to $2\ell'$ bits at the front. After eliminating the wrap-bit, for the number of right shift positions ζ , we left-shift $\hat{x}$ by $\ell' - \zeta$ positions, then keep the first ℓ' bits to obtain the result. Considering this method encounters issues when $\zeta > \ell'$, we need to perform one comparison between ζ and ℓ' . For cases where $\zeta > \ell'$, we set the right-shift result to 0. Finally, for the right-shift-by-one operation, since the shift amount "one bit" is public information, we directly employ a truncation protocol. The procedure involves computing the wrap-bit using Π_{wrap} , performing a local right shift by one bit, and then eliminating the wrap-bit. For details, please refer to Appendix A.7.

Table 2: Comparison of communication costs per operation (Bytes, batch size = 2^{18}), where $P_i \leftrightarrow P_j$ denotes the sum of communication cost sent from P_i to P_j and from P_j to P_i .

Operation	Secfloat [40]	Sasaki [42]	Ours			
			$P_0 \leftrightarrow P_1$	$P_0 \leftrightarrow P_2$	$P_1 \leftrightarrow P_2$	Total
FP Mul	2941	–	29	83	26	138
FP Add	10671	45377	252	667	470	1389

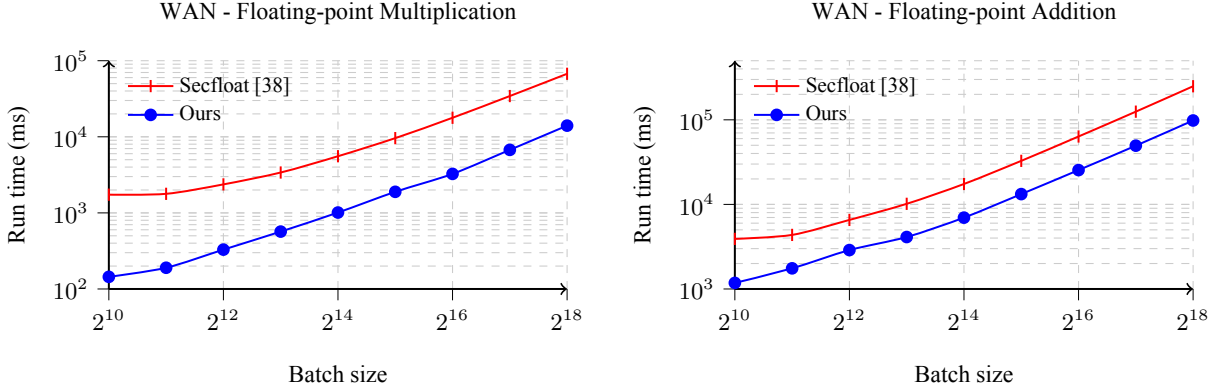


Figure 13: Performance of our floating-point protocols vs. Secfloat [38] across batch sizes. Left: multiplication; Right: addition.

6 Benchmarks

In this section, we benchmark our fixed-point and floating-point protocols. We compare our fixed-point protocol against the typical fixed-point solution [36, 45, 46, 11]. For floating-point operations, we compare with Secfloat [40]. We note that Secfloat is a 2PC protocol, while ours is a 3PC protocol. Unfortunately, we could not find any other open-source floating-point 3PC implementation for a meaningful comparison.

Table 3: Test Results from Floating Point Multiplication

Protocol	Run time (ms)	MAN setting			WAN setting		
		2^{10}	2^{14}	2^{18}	2^{10}	2^{14}	2^{18}
Secfloat (1 thread)	Total	318.63	2004.49	28648.13	1735.88	5555.41	67336.35
	Preprocess	16.45	115.18	1083.05	39.43	206.22	1366.40
Ours(1 thread)	Online	47.77	510.94	6916.59	104.93	804.38	12652.93
	Total	64.22	626.12	7999.64	144.36	1010.60	14019.33
Ours(16 threads)	Preprocess	27.22	42.80	545.57	22.91	102.24	1327.78
	Online	15.97	67.87	900.59	72.11	157.80	1890.34
	Total	43.19	110.67	1446.16	95.02	260.04	3218.12

6.1 Benchmark setting

We implement our protocols in C++. All the benchmark code [1] can be found on the anonymous GitHub repository. For our experiments, the benchmark is performed in a server that runs Ubuntu 18.04.2 LTS

Table 4: Test Results from Floating Point Addition

Protocol	Run time (ms)	MAN setting			WAN setting		
		2^{10}	2^{14}	2^{18}	2^{10}	2^{14}	2^{18}
Secfloat (1 thread)	Total	855.58	7501.77	113394.75	3911.68	17438.30	249510.73
	Preprocess	141.66	1516.39	13643.00	187.92	1571.33	15569.23
Ours(1 thread)	Online	478.63	4256.90	68045.48	991.67	5404.79	82652.53
	Total	620.29	5773.29	81688.48	1179.59	6976.12	98221.76
Ours(16 threads)	Preprocess	35.27	291.37	4419.64	51.57	645.41	9675.55
	Online	94.82	746.88	9506.06	554.05	1539.56	21770.42
	Total	130.09	1038.25	13925.70	605.62	2184.97	31445.97

with Intel(R) Xeon(R) Silver 4214 CPU @ 2.20GHz, 48 CPUs, 128 GB Memory. Our experiments are performed in a local area network, using traffic control in Linux to simulate three network settings: (1) metropolitan-area settings (MAN): 256Mbps bandwidth with 2ms round-trip latency (RTT). (2) wide-area setting (WAN): 100Mbps bandwidth with 20ms round-trip latency.

6.2 Experimental Results

Fixed-point Operations. Tables 1 presents a comparative performance analysis of our fixed-point multiplication. Fig. 12 illustrates the efficiency improvement factor of our fixed-point multiplication across different networks and data sizes. We compared the performance of our multiplication protocol against SOTA that use typical truncation methods (including ASTRA [12], FALCON [47], Bicaptor [52], Piranha [49], Orca [22], etc.). The experiments included test results for ℓ_i and ℓ_f ranging from 8 to 16, and data volumes spanning from 2^{16} to 2^{24} . For baselines, the corresponding truncation error probability is $2^{-\ell'-1}$. We set $\ell' = 32$ to ensure a sufficiently low probability of truncation errors.

The experimental results align with our expected theoretical values. For the case of $(\ell_i, \ell_f) = (8, 8)$, our online phase requires only a 2×16 bits cost, while the typical schemes require 2×64 bits, resulting in a $4 \times$ communication cost. Experiments further validate this ratio (for $(\ell_i, \ell_f) = (8, 8)$, WAN, and data size of 2^{24} , our scheme's time is approximately 1/4 of SOTA's). Our scheme demonstrates more significant advantages in low-bit multiplications. This is because $\ell' = 32$ is a fixed value, and when the multiplication bit width is small, ℓ' occupies a larger proportion, making our scheme more advantageous. However, as the multiplication bit width becomes large enough, the overhead introduced by ℓ' becomes a smaller proportion, and the advantage of our scheme diminishes. Moreover, taking advantage of the online phase, our overall overhead is also better than that of the typical truncation schemes. Overall, in conventional network scenarios, when the data size is large enough, our fixed-point multiplication scheme achieves performance in the online phase close to $4 \times$ that of typical truncation methods, with the overall performance approximately $1.6 \times$ that of the typical truncation methods.

Floating-point Operations. We conducted experiments on FP32, that is, setting $\ell_e = 8$ and $\ell_f = 23$. Table 2 depicts the actual communication overhead of our floating-point protocol compared to the Secfloat protocol. As shown, the communication overhead of our multiplication protocol is less than 5% of Secfloat's, while the overhead for addition is only 13% of Secfloat's. This offers a significant advantage in cloud computing scenarios, where communication costs are a major factor.

Tables 3 and 4 benchmark the running time of our floating-point primitives against Secfloat [40]. Fig. 13 illustrates the comparison of execution times in a WAN environment. Under single-threaded execution, our protocol consistently outperforms the baseline; specifically, for 2^{18} operations in a MAN environment, the total runtime for multiplication is recorded at 8 seconds for our protocol, significantly outperforming Secfloat's runtime of 28.6 seconds. Similarly, for floating point addition, our protocol completes in 81.7 seconds, compared to Secfloat's 113.4 seconds. To further assess the scalability of

our protocol, we employed 16 threads for parallel execution. This reduction in execution time is especially pronounced: for multiplication, the total runtime drops to 1.4 seconds, and for addition, the runtime reduces to 13.9 seconds. This dramatic reduction in time, compared to the single-threaded case, demonstrates the substantial throughput gains achievable with multi-threading.

These performance advantages are also evident in a WAN setting. When running on a WAN, the total time for multiplication with our protocol is 14.0 seconds and for addition is 98.2 seconds. In contrast, Secfloat’s performance under the same conditions is much slower, with multiplication taking 67.3 seconds and addition taking 249.5 seconds. The contrast is even more stark when 16 threads are used: for our protocol, the total runtimes drop to 3.2 seconds for multiplication and 31.4 seconds for addition. Our protocol demonstrates a clear advantage in high-delay networks, thanks to the constant-round nature of our design. Overall, our floating-point performance in multiplication is $5\times$ faster than Secfloat, and in addition, it is $3\times$ faster.

7 Conclusion

In this work, we address the issue of performing fixed-point multiplication over rings much larger than the bit width of the fixed-point numbers, and design constant-round floating-point protocols with low communication overhead. Experimental results show that our protocols achieve a $3 - 5\times$ performance improvement compared to the SOTA methods.

Ethical Implications

The development and deployment of privacy-preserving technologies, such as secure multi-party computation (MPC) and the proposed floating-point and fixed-point arithmetic protocols, have significant ethical implications. These protocols offer robust mechanisms to ensure the confidentiality of sensitive data while allowing collaborative computations. The proposed fixed-point and floating-point protocols adhere to ethical standards by ensuring the privacy of input values, as no sensitive data is disclosed during the process. Overall, this work contributes to improving privacy-preserving cryptographic protocols, supporting ethical data handling, and secure communications.

Compliance with Open Science Policy

We are committed to promoting transparency, reproducibility, and open access in scientific research. As part of our Open Science policy, we have made the code implementation of our protocols available via an anonymous link for peer review and community evaluation. We understand the importance of open access to research tools and encourage others to build upon our work. Upon acceptance of the paper, we will fully disclose the source code in a publicly accessible repository, along with detailed instructions for use and documentation. This will ensure that the community can validate our results and utilize our protocols for further research and practical applications. Additionally, we will submit our code for Artifacts Evaluation (AE) to ensure it meets standards for reproducibility, security, and performance. This process will help validate the correctness and usability of our work, contributing to the advancement of open science.

References

- [1] Our implementation code. <https://anonymous.4open.science/r/3-party-fixed-and-floating-point-09EE/>.
- [2] Mehrdad Aliasgari and Marina Blanton. Secure computation of hidden markov models. In *2013 International Conference on Security and Cryptography (SECRYPT)*, pages 1–12, 2013.

- [3] Mehrdad Aliasgari, Marina Blanton, and Fattaneh Bayatbabolghani. Secure computation of hidden markov models and secure floating-point arithmetic in the malicious model. *International Journal of Information Security*, 16(6):577–601, 2017.
- [4] Mehrdad Aliasgari, Marina Blanton, Yihua Zhang, and Aaron Steele. Secure computation on floating point numbers. *NDSS*, 2013.
- [5] Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 805–817, Vienna, Austria, October 24–28, 2016. ACM Press.
- [6] Donald Beaver. Efficient multiparty protocols using circuit randomization. In *CRYPTO*, 1991.
- [7] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In *EUROCRYPT*, 2011.
- [8] Loris Bergerat, Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. TFHE gets real: an efficient and flexible homomorphic floating-point arithmetic. Cryptology ePrint Archive, Report 2025/257, 2025.
- [9] Lennart Braun, Daniel Demmler, Thomas Schneider, and Oleksandr Tkachenko. Motion—a framework for mixed-protocol multi-party computation. *ACM Transactions on Privacy and Security*, 25(2):1–35, 2022.
- [10] Megha Byali, Harsh Chaudhari, Arpita Patra, and Ajith Suresh. Flash: Fast and robust framework for privacy-preserving machine learning. In *PoPETs*, 2020.
- [11] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. Astra: High throughput 3pc over rings with application to secure prediction. In *CCSW*, 2019.
- [12] Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. ASTRA: High throughput 3PC over rings with application to secure prediction. Cryptology ePrint Archive, Report 2019/429, 2019.
- [13] Harsh Chaudhari, Rahul Rachuri, and Ajith Suresh. Trident: Efficient 4pc framework for privacy preserving machine learning. In *NDSS*, 2020.
- [14] Yuntian Chen, Zhanyong Tang, Tianpei Lu, Bingsheng Zhang, Zhiying Shi, and Zheng Wang. Accelerating private large transformers inference through fine-grained collaborative computation. *IEEE Transactions on Information Forensics and Security*, 2025.
- [15] Daniel Demmler, Ghada Dessouky, Farinaz Koushanfar, Ahmad-Reza Sadeghi, Thomas Schneider, and Shaza Zeitouni. Automated synthesis of optimized circuits for secure computation. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, CCS ’15, page 1504–1517, New York, NY, USA, 2015. Association for Computing Machinery.
- [16] Ye Dong, Wen jie Lu, Xiaoyao Hou, Kang Yang, and Jian Liu. M&m: Secure two-party machine learning through efficient modulus conversion and mixed-mode protocols. Cryptology ePrint Archive, Paper 2025/1551, 2025.
- [17] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *STOC*, 1987.
- [18] Christopher Harth-Kitzerow, Ajith Suresh, and Georg Carle. Sok: Truncation untangled: Scaling fixed-point arithmetic for privacy-preserving machine learning to large models and datasets. *Proc. Priv. Enhancing Technol.*, 2025(4):369–391, 2025.

- [19] Brett Hemenway, Steve Lu, Rafail Ostrovsky, and William Welser, IV. High-precision secure computation of satellite collision probabilities. In Vassilis Zikas and Roberto De Prisco, editors, *SCN 16: 10th International Conference on Security in Communication Networks*, volume 9841 of *Lecture Notes in Computer Science*, pages 169–187, Amalfi, Italy, August 31 – September 2, 2016. Springer, Cham, Switzerland.
- [20] Wilko Henecka, Stefan Kögl, Ahmad-Reza Sadeghi, Thomas Schneider, and Immo Wehrenberg. Tasty: Tool for automating secure two-party computations. In *CCS*, 2010.
- [21] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. Orca: Fss-based secure training and inference with gpus. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 597–616, 2024.
- [22] Neha Jawalkar, Kanav Gupta, Arkaprava Basu, Nishanth Chandran, Divya Gupta, and Rahul Sharma. Orca: FSS-based secure training and inference with GPUs. In *2024 IEEE Symposium on Security and Privacy*, pages 597–616, San Francisco, CA, USA, May 19–23, 2024. IEEE Computer Society Press.
- [23] Keyu Ji, Bingsheng Zhang, Tianpei Lu, Lichun Li, and Kui Ren. Uc secure private branching program and decision tree evaluation. *IEEE Transactions on Dependable and Secure Computing*, 20(4):2836–2848, 2023.
- [24] Marcel Keller and Ke Sun. Secure quantized training for deep learning. In *International Conference on Machine Learning*, pages 10912–10938. PMLR, 2022.
- [25] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. Crypten: Secure multi-party computation meets machine learning. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 4961–4973. Curran Associates, Inc., 2021.
- [26] Nishant Kumar, Mayank Rathee, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. CryptFlow: Secure TensorFlow inference. In *2020 IEEE Symposium on Security and Privacy*, pages 336–353, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press.
- [27] Yun Li, Yufei Duan, Zhicong Huang, Cheng Hong, Chao Zhang, and Yifan Song. Efficient 3PC for binary circuits with application to maliciously-secure DNN inference. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 5377–5394, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.
- [28] Fengrun Liu, Xiang Xie, and Yu Yu. Scalable multi-party computation protocols for machine learning in the honest-majority setting. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024: 33rd USENIX Security Symposium*, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association.
- [29] Tianpei Lu, Xin Kang, Bingsheng Zhang, Zhuo Ma, Xiaoyuan Zhang, Yang Liu, Kui Ren, and Chun Chen. Efficient 2pc for constant round secure equality testing and comparison. 2025.
- [30] Tianpei Lu, Bingsheng Zhang, Hao Li, and Kui Ren. A new paradigm for privacy-preserving decision tree evaluation. *Cryptology ePrint Archive*, Paper 2025/1447, 2025.
- [31] Tianpei Lu, Bingsheng Zhang, Lichun Li, and Kui Ren. The Communication-Friendly Privacy-Preserving Machine Learning against Malicious Adversaries. *IEEE Transactions on Dependable and Secure Computing*, (01):1–14, October 5555.
- [32] Tianpei Lu, Bingsheng Zhang, Lichun Li, Yuzhou Zhao, and Kui Ren. Lightning fast secure comparison for 3pc ppml. *Cryptology ePrint Archive*, 2023.

- [33] Tianpei Lu, Bingsheng Zhang, Xiaoyuan Zhang, and Kui Ren. A new PPML paradigm for quantized models. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025.
- [34] Payman Mohassel and Peter Rindal. Aby3: A mixed protocol framework for machine learning. In *CCS*, 2018.
- [35] Arpita Patra and Ajith Suresh. BLAZE: blazing fast privacy-preserving machine learning. In *NDSS*, 2020.
- [36] Mohassel Payman and Zhang Yupeng. Secureml: A system for scalable privacy-preserving machine learning. In *S&P*, 2017.
- [37] Pille Pullonen and Sander Siim. Combining secret sharing and garbled circuits for efficient private IEEE 754 floating-point computations. In Michael Brenner, Nicolas Christin, Benjamin Johnson, and Kurt Rohloff, editors, *FC 2015 Workshops*, volume 8976 of *Lecture Notes in Computer Science*, pages 172–183, San Juan, Puerto Rico, January 30, 2015. Springer Berlin Heidelberg, Germany.
- [38] Deevashwer Rathee, Anwesh Bhattacharya, Divya Gupta, Rahul Sharma, and Dawn Song. Secure {Floating-Point} training. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6329–6346, 2023.
- [39] Deevashwer Rathee, Anwesh Bhattacharya, Divya Gupta, Rahul Sharma, and Dawn Song. Secure floating-point training. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 6329–6346, Anaheim, CA, USA, August 9–11, 2023. USENIX Association.
- [40] Deevashwer Rathee, Anwesh Bhattacharya, Rahul Sharma, Divya Gupta, Nishanth Chandran, and Aseem Rastogi. Secfloat: Accurate floating-point meets secure 2-party computation. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 576–595. IEEE, 2022.
- [41] M. Sadegh Riazi, Christian Weinert, Oleksandr Tkachenko, Ebrahim M. Songhori, Thomas Schneider, and Farinaz Koushanfar. Chameleon: A hybrid secure computation framework for machine learning applications. In *ASIACCS*, 2018.
- [42] Kota Sasaki and Koji Nuida. Efficiency and accuracy improvements of secure floating-point addition over secret sharing. In Kazumaro Aoki and Akira Kanaoka, editors, *IWSEC 20: 15th International Workshop on Security, Advances in Information and Computer Security*, volume 12231 of *Lecture Notes in Computer Science*, pages 77–94, Fukui, Japan, September 2–4, 2020. Springer, Cham, Switzerland.
- [43] Sijun Tan, Brian Knott, Yuan Tian, and David J. Wu. Cryptgpu: Fast privacy-preserving machine learning on the gpu. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1021–1038, 2021.
- [44] Sameer Wagh, Divya Gupta, and Nishanth Chandran. SecureNN: Efficient and private neural network training. Cryptology ePrint Archive, Report 2018/442, 2018.
- [45] Sameer Wagh, Divya Gupta, and Nishanth Chandran. Securenn: 3-party secure computation for neural network training. In *PoPETs*, 2019.
- [46] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. Falcon: Honest-majority maliciously secure framework for private deep learning. In *PoPETs*, 2021.
- [47] Sameer Wagh, Shruti Tople, Fabrice Benhamouda, Eyal Kushilevitz, Prateek Mittal, and Tal Rabin. Falcon: Honest-majority maliciously secure framework for private deep learning. *Proceedings on Privacy Enhancing Technologies*, 2021(1):188–208, January 2021.

- [48] Cong Wang, Kui Ren, Jia Wang, and Qian Wang. Harnessing the cloud for securely outsourcing large-scale systems of linear equations. *IEEE Transactions on Parallel and Distributed Systems*, 24(6):1172–1181, 2013.
- [49] Jean-Luc Watson, Sameer Wagh, and Raluca Ada Popa. Piranha: A gpu platform for secure computation, 2022.
- [50] Andrew C. Yao. Protocols for secure computations. In *SFCS*, 1982.
- [51] Lindell Yehuda and Nof Ariel. A framework for constructing fast mpc over arithmetic circuits with malicious adversaries and an honest-majority. In *CCS*, 2017.
- [52] Lijing Zhou, Ziyu Wang, Hongrui Cui, Qingrui Song, and Yu Yu. Bicoprot: Two-round secure three-party non-linear computation without preprocessing for privacy-preserving machine learning. In *2023 IEEE Symposium on Security and Privacy*, pages 534–551, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.
- [53] Lijing Zhou, Bingsheng Zhang, Ziyu Wang, Tianpei Lu, Qingrui Song, Su Zhang, Hongrui Cui, and Yu Yu. On probabilistic truncation in privacy-preserving machine learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(21):22955–22964, Apr. 2025.

A Other Building Blocks.

A.1 Sign-bit Extraction Protocol Proposed by Lu *et al.* [32]

Fig. 14 depicts sign-bit extraction protocol. It accepts a share $\langle x \rangle^M$ over the ring $\mathbb{Z}_M$ and outputs a share $\langle z \rangle^K$ over the ring $\mathbb{Z}_K$. It requires $2\ell \cdot \log \ell + 2 \log K$ bits communication cost in the online phase and $\ell \cdot \log \ell + 2 \log K$ bits communication cost in the offline phase for $\ell = \lceil \log M \rceil$. For the sign protocol, we can simultaneously generate $[r'_1]^{K_1}, \dots, [r'_n]^{K_n}, [r_{x,1}]^{K_1}, \dots, [r_{x,n}]^{K_n}$, and $[\Gamma_1]^{K_1}, \dots, [\Gamma_n]^{K_n}$ on different rings. After P_0 computes t , we secret-share $m'_1 = t - r'_1, \dots, m'_n = t - r'_n$, thereby obtaining the secret shares of z over different rings. We denote the protocol that outputs shares over n rings as $\Pi_{\text{sign}}^{M, K_1, \dots, K_n}$. Its online cost is 2 rounds with $2\ell \cdot \log \ell + 2 \sum_{i=1}^n \log K_i$ bits of communication, and the offline cost is $\ell \cdot \log \ell + 2 \sum_{i=1}^n \log K_i$ bits.

A.2 Wrap-bit Evaluation Protocol Π_{wrap}

In this section, we realize the wrap-bit evaluation protocol $\langle z \rangle^K \leftarrow \Pi_{\text{wrap}}^{M, K}(\langle x \rangle^K)$. Our wrap-bit protocol is divided into two categories. For $\langle x \rangle := (m_x, [r_x])$: The first category only computes the wrap-bit for the portion $x = m_x + r_x$, denoted by Π_{wrap1} . The second category computes the overall wrap-bit for $x = m_x + [r_x]_1 + [r_x]_2$, denoted by Π_{wrap2} .

For $x < 2^{\lceil \log M \rceil - 1}$, namely, the first bit of x is 0, we calculate the wrap-bit of $m_x + r_x$ using $z = m_{x,0} \vee r_{x,0}$. For the wrap-bit of $[r_x]_1 + [r_x]_2$, since P_0 know $[r_x]_1$ and $[r_x]_2$, we let P_0 direct calculate it. Fig. 15 depicts our wrap-bit evaluation protocol $\Pi_{\text{wrap1}}^{M, K}$ for $x < 2^{\lceil \log M \rceil - 1}$. For $\Pi_{\text{wrap1}}^{M, K}$, we can have protocol $\Pi_{\text{wrap2}}^{M, K}$ directly achieve it by omitting the computation of the wrap-bit for the $[r_x]_1 + [r_x]_2$ part.

When the first bit of x cannot be guaranteed to be 0, we need to pad $\langle x \rangle$ with a prefix zero, then run Π_{sign} on that to obtain $\langle b \rangle$. Considering that we aim to calculate the wrap-bit d of $m_x + r_x = x \pmod{M}$ and P_0 knows the wrap bit c of $r_x = [r_x]_1 + [r_x]_2$, we have $b = c \oplus d$. Then, for the wrap bit e of $[r_x]_1 + [r_x]_2 + m_x$, we have $e = d + c$. After obtaining b in this way, we can calculate $d = b \oplus c = b + c - 2bc$ using Π_{mult} as the output of $\Pi_{\text{wrap1}}^{M, K}$, and then compute $e = d + c$ as the output of $\Pi_{\text{wrap2}}^{M, K}$. Fig. 16 depicts our wrap-bit evaluation protocol $\Pi_{\text{wrap2}}^{M, K}$ for an arbitrary x .

Efficiency. For the case where $x < 2^{\lceil \log M \rceil - 1}$, Protocol $\Pi_{\text{wrap1}}^{M, K}$ requires $\log K$ bits of communication in the offline phase and 1 round with $2 \log K$ bits of communication in the online phase. Protocol $\Pi_{\text{wrap2}}^{M, K}$

Protocol $\Pi_{\text{sign}}^{M,K}(\langle x \rangle^M)$

P_j and P_k hold the common seed $\eta_{j,k} \in \{0,1\}^\lambda$. $\ell = \lceil \log M \rceil$

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of $z = \text{sign}(x)$.

Preprocessing:

- All parties perform $[r']^K, [r_z]^K \leftarrow \Pi_{[\cdot]}^K$; Let P_0 know $[r']^K$ and $[r_z]^K$.
- P_i , for $i \in \{1, 2\}$ generates the same random value $\Delta \in \{0, 1\}$ via PRF with seed $\eta_{1,2}$ and reveals $[\Gamma]^K = \Delta + [r']^K - 2\Delta \cdot [r']^K + [r_z]^K$ to each other.
- P_0 does:
 1. calculate $\hat{r}_x = r_x - \text{sign}(r_x) \cdot 2^{\ell-1} \in \mathbb{Z}_{2^{\ell-1}}$
 2. extract $\hat{r}_x$ as $\{r_{x,0}, \dots, r_{x,\ell-2}\}$
 3. perform $[r_{x,i}]^p \leftarrow \Pi_{[\cdot]}^p(r_{x,i})$ for $i \in \mathbb{Z}_{\ell-1}$, taking the smallest prime p bigger than $\log M$;

Online:

- P_j , for $j \in \{1, 2\}$ does:
 1. set $\hat{m}_x = m_x - \text{sign}(m_x) \cdot 2^{\ell-1}$ and bitexact it as $\{\hat{m}_{x,i} \in \{0, 1\}\}_{i \in \mathbb{Z}_{\ell-1}}$ while $\sum_{i=0}^{\ell-2} 2^{\ell-2-i} \hat{m}_{x,i} = \hat{m}_x$;
 2. set $\hat{m}_{x,\ell-1} = 1$ and $[r_{x,\ell-1}]^p = [0]^p$;
 3. set $[m_i]^p = \hat{m}_{x,i} + [r_{x,i}]^p - 2\hat{m}_{x,i} \cdot [r_{x,i}]^p$ for $i \in \mathbb{Z}_\ell$.
 4. pick same random values $\{w_i\}_{i \in \mathbb{Z}_\ell} \in (\mathbb{Z}_p^*)^{2^\ell}$ via PRF with seed $\eta_{1,2}$;
 5. calculate $[m'_i]^p = \sum_{t=1}^i [m_t]^p - 2 \cdot [m_i]^p + 1$ and $[u_i]^p = w_i \cdot [m'_i]^p \cdot (1 \oplus \text{sign}(m_x) \oplus \hat{m}_{x,i} \oplus \Delta) + w_i \cdot (\text{sign}(m_x) \oplus \hat{m}_{x,i} \oplus \Delta)$ for $i \in \mathbb{Z}_\ell$;
 6. pick a random permutation π via PRF with seed $\eta_{1,2}$ and permute the list $\{[u_i]^p\}_{i \in \mathbb{Z}_\ell} = \pi(\{[u_i]^p\}_{i \in \mathbb{Z}_\ell})$;
 7. reveal $\{[u_i]^p\}_{i \in \mathbb{Z}_\ell}$ to P_0 ;
- P_0 sets $t = \text{sign}(r_x)$ if $\exists u_i = 0$ for $i \in \mathbb{Z}_\ell$ else $t = 1 \oplus \text{sign}(r_x)$ to P_j , for $j \in \{1, 2\}$; %output binary share (Δ, t)
- P_0 sends $m' = t - r'$ to P_1 and P_2 ;
- P_j , for $j \in \{1, 2\}$ sets $m_z = m' - 2\Delta \cdot m' + \Gamma$;
- Output $(m_z, [r_z]^K)$.

Figure 14: Sign-bit Extraction Protocol.

requires $2 \log K$ bits of communication in the offline phase and 1 round with $2 \log K$ bits of communication in the online phase. For an arbitrary range of x , both $\Pi_{\text{wrap1}}^{M,K}$ and $\Pi_{\text{wrap2}}^{M,K}$ require $\ell \cdot \log \ell + 2 \log K$ bits of communication in the offline phase and 2 rounds (The work by Lu *et al.* [32] mentions that for the output $\langle d \rangle$, if a multiplication operation is subsequently required, its round can be merged Π_{sign} to eliminate the need for a dedicated multiplication protocol round. Please refer to their Appendix A.1 for details.) with $2\ell \cdot \log \ell + 4 \log K$ bits of communication in the online phase.

A.3 Wrap-bit and Sign-bit Evaluation Protocol $\Pi_{\text{wrap/sign}}$

We observe that when both the sign bit and the wrap bit of (x) need to be computed, the wrap bit can be determined from the sign bit, making the computation significantly cheaper than evaluating the wrap bit directly. Specifically, consider $x = m_x + r_x$. When the sign bit is 0, as described in Section 3, the wrap bit can be obtained as $m_{x,(0)} \vee r_{x,(0)}$. When the sign bit is 1, the wrap bit becomes $m_{x,(0)} \wedge r_{x,(0)}$. The reason is that if $(m_{x,(0)}, r_{x,(0)}) = (1, 0)$, $(0, 1)$, or $(0, 0)$, no additional carry can occur that would make the sign bit 1; therefore, the wrap bit must be 0. Only in the case $(1, 1)$ can the wrap bit be 1. So we can calculate wrap bit with $m_{x,(0)} \wedge r_{x,(0)}$. Thus, using the sign bit together with $m_{x,(0)}$ and $r_{x,(0)}$, we can

Protocol $\Pi_{\text{wrap2}}^{M,K}(\langle x \rangle^M)$

x meets $x_{(0)} = 0$.

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of z , while $x + z \cdot M = m_x + [r_x]_1 + [r_x]_2$.

Preprocessing:

- All parties perform $[r_z]^K \leftarrow \Pi_{[\cdot]}^K$; Let P_0 know $[r_z]^K$.
- P_0 extract $r_{x,(0)}$ and invoke $[r_{x,(0)}]^K \leftarrow \Pi_{[\cdot]}^K(r_{x,(0)})$:
- P_0 calculate $b = 1 - (r_x \stackrel{?}{=} [r_x]_1 + [r_x]_2)$ and invoke $[b]^K \leftarrow \Pi_{[\cdot]}^K(b)$:

Online:

- P_j , for $j \in \{1, 2\}$ does:
 1. calculate $[m_z] = m_{x,(0)} + [b]^K + [r_{x,(0)}]^K - m_{x,(0)}[r_{x,(0)}]^K - [r_z]^K$;
 2. reveal m_z ;
- Output $(m_z, [r_z]^K)$.

Figure 15: Wrap-bit Evaluation Protocol.

Protocol $\Pi_{\text{wrap2}}^{M,K}(\langle x \rangle^M)$

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of z , while $x + z \cdot M = m_x + [r_x]_1 + [r_x]_2$.

Preprocessing:

- P_0 calculate $c = 1 - (r_x \stackrel{?}{=} [r_x]_1 + [r_x]_2)$ and invoke $\langle c \rangle^K \leftarrow \Pi_{\langle \cdot \rangle}^K(c)$:

Online:

- All parties perform $\langle b \rangle^K \leftarrow \Pi_{\text{sign}}^{2M,K}(\langle x \rangle)$;
- All parties perform Π_{mult}^K to calculate $\langle d \rangle = \langle b \rangle + \langle c \rangle - 2\langle b \rangle \cdot \langle c \rangle$; % $\langle d \rangle$ is the result of Π_{wrap1} ;
- Output $\langle z \rangle = \langle d \rangle + \langle c \rangle$

Figure 16: Wrap-bit Evaluation Protocol.

correctly compute the wrap bit. Fig. 17 illustrates our protocol.

Efficiency. Taking $\ell = \log M$, $\Pi_{\text{wrap2/sign}}^{M,K,Q}(\langle x \rangle^M)$ requires $\ell \log \ell + 2 \log Q + 4 \log K$ bits in the offline phase and 4 rounds $2\ell \log \ell + 2 \log Q + 8 \log K$ bits.

A.4 Secure Comparison Protocol Π_{great}

For value $\langle x \rangle^M$ and $\langle y \rangle^M$, its comparison $\Pi_{\text{great}}^{M,K}$ can be calculated by sign. It holds that $\text{great}(x, y) = (1 \oplus \text{sign}(x) \oplus \text{sign}(y)) \cdot \text{sign}(y - x) + (\text{sign}(x) \oplus \text{sign}(y)) \cdot \text{sign}(y)$. We sequentially invoke $\Pi_{\text{sign}}^{M,K}$ to evaluate $a = \text{sign}(x)$, $b = \text{sign}(y)$, and $c = \text{sign}(y - x)$, and then compute the aforementioned expression to obtain $\langle \text{great}(x, y) \rangle^K$.

Efficiency. $\Pi_{\text{great}}^{M,K}$ requires $3\ell \cdot \log \ell + 5 \log K$ bits of communication in the offline phase and 3 rounds with $6\ell \cdot \log \ell + 6 \log K$ bits of communication in the online phase.

Protocol $\Pi_{\text{wrap2/sign}}^{M,K,Q}(\langle x \rangle^M)$

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of wrap bit z and $\langle \cdot \rangle^Q$ -shared value of sign bit c .

Preprocessing:

- All parties perform $[r_{z_1}]^K \leftarrow \Pi_{[\cdot]}^K$; Let P_0 know $[r_{z_1}]^K$ and $[r_{z_2}]^K$.
- P_0 extract $r_{x,(0)}$ and invoke $[r_{x,(0)}]^K \leftarrow \Pi_{[\cdot]}^K(r_{x,(0)})$:
- P_0 calculate $b = 1 - (r_x \stackrel{?}{=} [r_x]_1 + [r_x]_2)$ and invoke $[b]^K \leftarrow \Pi_{[\cdot]}^K(b)$:

Online:

- All parties perform $(\langle c \rangle^K, \langle c \rangle^Q) \leftarrow \Pi_{\text{sign}}^{M,K,Q}(\langle x \rangle)$;
- P_j , for $j \in \{1, 2\}$ does:
 1. calculate $[m_{z_1}] = m_{x,(0)} + [b]^K + [r_{x,(0)}]^K - m_{x,(0)}[r_{x,(0)}]^K - [r_{z_1}]^K$;
 2. calculate $[m_{z_2}] = [b]^K + m_{x,(0)}[r_{x,(0)}]^K - [r_{z_2}]^K$;
 3. reveal m_{z_1} and m_{z_2} ;
 4. invoke Π_{mult} to calculate $\langle z \rangle^K = \langle c \rangle^K \cdot (\langle z_1 \rangle^K - \langle z_2 \rangle^K) + \langle z_2 \rangle^K$;
- Output $(\langle z \rangle^K, \langle c \rangle^Q)$.

Figure 17: Wrap-bit and Sign-bit Evaluation Protocol.

A.5 Secure Equality Testing Protocol Π_{eq}

Lu *et al.* [29] implemented a secure equality test for two parties by checking the Hamming weight. Following their approach, we have extended this to a three-party setting. For secret-shared values $\langle x \rangle^M$ and $\langle y \rangle^M$, their equality corresponds to $x - y = 0$. We begin by computing $\langle \Delta \rangle^M = \langle x \rangle^M - \langle y \rangle^M$. The condition $\Delta = 0$ is equivalent to $m_\Delta = -r_\Delta$. In the offline phase, P_0 secretly shares $-r_\Delta$ bitwise. In the online phase, P_1 and P_2 can locally compute $[\delta_i] = m_{\Delta,(i)} \oplus [r_{\Delta,(i)}]$ for each i^{th} bit. The sum $[\delta] = \sum [\delta_i]$ equals zero if and only if $\Delta = 0$. To prevent wrap-around from affecting the zero result, we perform this computation modulo $\log M$. Through this computation, the original problem of determining whether $\langle \Delta \rangle^M$ equals zero is transformed into testing whether $[\delta]^{\log M}$ is zero. We can then iteratively convert $[\delta]^{\log M}$ back into $\langle \delta \rangle^{\log M}$ for multiple rounds of dimension reduction. Once the value is reduced to a sufficiently small ring, denoted by $\mathbb{Z}_{M'}$, yielding $[\zeta]^{M'}$, we instruct P_0 to secretly share a lookup table $T = (t_0, \dots, t_{M'-1})$ and a value $r \in \mathbb{Z}_{M'}$ in the offline phase, where $t_r = 1$ and all other $t_i = 0$. P_1 and P_2 then open $\delta' = [\delta]^{M'} + [r]^{M'}$ and locally set $[z] = [T_{\delta'}]$. Fig. 18 depicts our secure equality testing protocol.

Efficiency. Taking $\ell = \log M$, $\Pi_{\text{eq}}^{M,K}$ requires $\ell \cdot \log \ell + \log K \cdot (\ell + 1)$ bits of communication in the offline phase and 2 rounds with $2 \log \ell + 2 \log K$ bits of communication in the online phase.

A.6 Secure Selection Protocol

We realize the secure selection protocol Π_{select} . This protocol takes as input a secret-shared bit $\langle b \rangle^2$ over $\mathbb{Z}_2$, along with two secret-shared vectors $\langle X \rangle$ and $\langle Y \rangle$ over arbitrary rings. Based on the value of b , Π_{select} outputs either $\langle X \rangle$ or $\langle Y \rangle$. Fig. 19 illustrates the construction of the secure selection protocol for a single element.

Efficiency. Π_{select}^M requires $\log M$ bits of communication in the offline phase and 2 rounds with $4 \log M$ bits of communication in the online phase.

Protocol $\Pi_{\text{eq}}^{M,K}(\langle x \rangle^M, \langle y \rangle^M)$

Taking the smallest prime p bigger than $\log M$.

Input : $\langle \cdot \rangle^M$ -shared value of x and y .

Output : $\langle \cdot \rangle^K$ -shared value of z , while $z = x \stackrel{?}{=} y$.

Preprocessing:

- P_0 picks random value $\sigma \in \mathbb{Z}_p$, and set $T = (t_0, \dots, t_{p-1})$, while $t_\sigma = 1$ and $t_i = 0$ for $i \neq \sigma$;
- P_0 set $r_\Delta = r_x - r_y \pmod{M}$;
- P_0 bitextract $r = -r_\Delta$ and secret share them as $([r_{(0)}], \dots, [r_{(\log M - 1)}]) \leftarrow \Pi_{[\cdot]}^P(r_{(0)}, \dots, r_{(\log M - 1)})$;
- All parties invoke $[T]^K \leftarrow \Pi_{[\cdot]}^K(T)$, $[\sigma]^P \leftarrow \Pi_{[\cdot]}^P(\sigma)$ and $[r_z]^K \leftarrow \Pi_{[\cdot]}^K$;

Online:

- All parties perform $\langle \Delta \rangle^M = \langle x \rangle^M - \langle y \rangle^M$;
- P_1 and P_2 calculate $[\delta]^P = \sum_{i=0}^{\log M - 1} m_{\Delta, (i)} + [r_{(i)}] - 2m_{\Delta, (i)}[r_{(i)}]$;
- P_1 and P_2 calculate and reveal $[\delta']^P = [\delta]^P + [\sigma]^P$;
- P_1 and P_2 calculate and reveal $[m_z]^K = [t_{\delta'}]^K - [r_z]^K$;
- Output $(m_z, [r_z]^K)$

Figure 18: Secure Equality Testing Protocol.

Protocol $\Pi_{\text{select}}^M(\langle b \rangle^2, \langle x \rangle^M, \langle y \rangle^M)$

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of z , while $z = x$ if $b = 1$ and $z = y$ if $b = 0$.

Preprocessing:

- P_0 invoke $\langle r_b \rangle^M \leftarrow \Pi_{\langle \cdot \rangle}^M(r_b)$;

Online:

- All parties set $\langle m \rangle^M = (m_b, [0]^M)$;
- All parties perform Π_{mult}^M to calculate $\langle b' \rangle^M = \langle m \rangle^M + \langle r_b \rangle^M - 2\langle m \rangle^M \cdot \langle r_b \rangle^M$;
- All parties perform Π_{mult}^M to calculate $\langle z \rangle = \langle b' \rangle(\langle x \rangle - \langle y \rangle) + \langle y \rangle$;

Figure 19: Secure Selection Protocol.

A.7 Secure Truncation Protocol

We realize the secure truncation protocol $\Pi_{\text{trunc}}^{M,\delta}$. It accepts input of shared value $\langle x \rangle$ and outputs truncated result $\langle z \rangle$ while $z = \lfloor x/2^\delta \rfloor$. The protocol first invokes the wrap-bit protocol $\Pi_{\text{wrap2}}^{M,M}$ to obtain the wrap value of $\langle x \rangle$. Then, the secret shares of $\langle x \rangle$ are locally right-shifted by δ bits. Since this operation shifts the original wrap value to the $(\delta - 1)^{\text{th}}$ bit of the result, we eliminate this wrap value from the shifted $\langle x \rangle$ to produce the final output. Fig. 20 depicts our truncation protocol.

Protocol $\Pi_{\text{trunc}}^{M,\delta}(\langle x \rangle^M)$

Input : $\langle \cdot \rangle^M$ -shared value of x .

Output : $\langle \cdot \rangle^K$ -shared value of z , while $z = \text{cut}(z, \delta)$.

Execution:

- All parties perform $\langle b \rangle^M = \Pi_{\text{wrap2}}^{M,M}(\langle x \rangle^M)$;
- All parties set $\langle x' \rangle^M := (\lfloor m_x/2^\delta \rfloor, \lfloor [r_x]_1/2^\delta \rfloor, \lfloor [r_x]_2/2^\delta \rfloor)$;
- All parties calculate $\langle z \rangle^M := \langle x' \rangle^M - 2^{\lceil \log M \rceil - \delta} \cdot \langle b \rangle^M$;

Figure 20: Secure Truncation Protocol.

B Security Proof

B.1 Proof of Theorem 1

Theorem 1. *The protocol $\Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}$ as depicted in Fig. 2 secure compute $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$ in the $\mathcal{F}_{\text{share}}$ -hybrid model against semi-honest PPT adversaries who can statically corrupt up to one party.*

Proof. We first prove the correctness, namely, the output of $\Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}$ is same as $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$.

It is easy to verify that for protocol $\Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}$, the secret sharing $[z]^{\ell' + \ell_f}$ where $z = x \cdot y$, we need to examine whether the truncated value $[z']^{\ell'}$ is consistent with the $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$. This can be simply verified by checking the consistency of the distribution.

- First, the secret sharing shares of $[z]^{\ell' + \ell_f}$ can be considered as random numbers, because $[z]^{\ell' + \ell_f}$ is obtained by adding $[\Gamma]$, which is generated by secret sharing. Therefore, the shares $[z]_1^{\ell' + \ell_f}$ or $[z]_2^{\ell' + \ell_f}$ of $[z]^{\ell' + \ell_f}$ can be treated as random numbers.
- For the random secret shares $[z]_1^{\ell' + \ell_f}$ and $[z]_2^{\ell' + \ell_f}$ of $[z]^{\ell' + \ell_f}$, we can analyze the distribution of the result after dropping the lower ℓ_f bits. Let the last ℓ_f bits of $[z]_1$ be denoted as $[z_f]_1$, with the corresponding last ℓ_f bits of $[z]_2$ as $[z_f]_2$. We have the following: when $[z_f]_1 + [z_f]_2 < 2^{\ell_f}$, $z' = [z]$; when $[z_f]_1 + [z_f]_2 \geq 2^{\ell_f}$, $z' = [z] - 1$.
- Analyzing the probability, for a randomly chosen $[z_f]_1$, it has a probability of $\frac{z_f + 1}{2^{\ell_f}}$ of lying within the interval $[0, z_f]$. In this case, $[z_f]_1 \leq z_f$ and no wrap-around occurs, so $z' = [z]$; otherwise, $[z_f]_1$ has a probability of $1 - \frac{z_f + 1}{2^{\ell_f}}$ of lying in the range $(z_f, 2^{\ell_f})$, in which case wrap-around occurs.

From this, we conclude that for protocol $\Pi_{\text{fix-mult}}^{\ell_i, \ell_f, \ell'}$, the output distribution is consistent with the functionality.

Then we construct a PPT simulator $\mathcal{S}$, such that no non-uniform PPT environment $\mathcal{Z}$ can distinguish between the ideal world and the real world. We consider the following cases:

Simulator for P_0 :

- $\mathcal{S}$ simulates $\mathcal{F}_{\text{share}}$ internally. $\mathcal{S}$ sends share r_x and r_y to $\mathcal{F}_{\text{fix-mult}}[\ell_f]$;
- Upon receiving $([r_z]_1, [r_z]_2)$ from $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$, $\mathcal{S}$ records $([r_z]_1, [r_z]_2)$;
- When corrupted P_0 invoke $[r_z] \leftarrow \mathcal{F}_{\text{share}}[2^{\ell'}]$, $\mathcal{S}$ sends $([r_z]_1, [r_z]_0)$ to corrupted P_0 .

Indistinguishability. Since P_0 only obtains $[r_z]_1$ and $[r_z]_2$ from $\mathcal{F}_{\text{share}}$, it is clear that the ideal world and the real world are indistinguishable.

Simulator for P_1 (or P_2):

- $\mathcal{S}$ simulates $\mathcal{F}_{\text{share}}$ internally. $\mathcal{S}$ sends share $m_x, [r_x]_1$ and $m_y, [r_y]_1$ to $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$;
- Upon receiving $([r_z]_1, m_z)$ from $\mathcal{F}_{\text{fix-mult}}[\ell_i, \ell_f, \ell']$, $\mathcal{S}$ records $([r_z]_1, m_z)$;
- When corrupted P_0 invoke $[r_z] \leftarrow \mathcal{F}_{\text{share}}[\ell']$, $\mathcal{S}$ sends $([r_z]_1)$ to corrupted P_1 .
- $\mathcal{S}$ calculates $[z]_1^{\ell' + \ell_f}, [z]_1^{\ell'} - [r_z]_1$ and set $\delta = m_z - ([z]_1^{\ell'} - [r_z]_1)$;
- $\mathcal{S}$ plays the role of P_2 to send δ to corrupted P_1 .

Indistinguishability. The message received by P_1 is $[r_z]_1$ output by $\mathcal{F}_{\text{share}}$ and δ , and by using δ , m_z is calculated. It can be verified that m_z is consistent with the output of the functionality. It is clear that the ideal world and the real world are indistinguishable.

This concludes the proof. $\square$

B.2 Proof of Theorem 2

Theorem 2. Let $x = m + r \pmod{2^\ell}$ with the binary representation $x = x_{(0)} \| x_{(1)} \| \dots \| x_{(\ell-1)}$ ($x_{(0)}$ is the most significant bit) be a non-zero value, and let ζ be the position of the first non-zero bit of x , i.e.,

$$x_{(\zeta)} = 1 \wedge \forall i < \zeta : x_{(i)} = 0$$

Let $b \in \{0, 1\}$ be the wrap-bit such that $m + r = 2^\ell \cdot b + x$. If $b = 1$, there exists an index σ such that $m_{(i)} \wedge r_{(i)} = 0$ for $i < \sigma$ and $m_{(\sigma)} \wedge r_{(\sigma)} = 1$. If we define m' and r' by setting $m'_{(i)} = 0, r'_{(i)} = 0$ for all $i \leq \sigma$ (and $m'_{(i)} = m_{(i)}, r'_{(i)} = r_{(i)}$ for all $i > \sigma$), then $m' + r' = x$ and the corresponding wrap-bit $b' = 0$. If $b = 0$, there exists an index σ such that $m_{(\sigma)} \vee r_{(\sigma)} = 1$ and $m_{(i)} \vee r_{(i)} = 0$ for $i < \sigma$. Then ζ can only be either σ or $\sigma - 1$.

Proof. First, we prove the case where $b = 1$. We begin by proving the existence of σ . If σ does not exist, then for all i , $m_{(i)} \wedge r_{(i)} = 0$. Let $c_{(i)}$ denote the carry to the i -th bit. Then, $m_{(\ell-1)} + r_{(\ell-1)} < 2$, meaning no carry is generated, and $c_{(\ell-2)} = 0$. Similarly, $m_{(\ell-1)} + r_{(\ell-1)} + c_{(\ell-2)} < 2$, again no carry is generated. By recursion up to the highest bit $m_{(0)} + r_{(0)} + c_{(02)} < 2$, no carry is produced, so $b = 0$, which contradicts the assumption. Therefore, σ must exist.

Next, we construct an auxiliary value $\delta = \sum_{i=0}^{\sigma} 2^{\ell-1-i} m_{(i)} + r_{(i)}$. Since $x = \sum_{i=0}^{\ell-1} 2^{\ell-1-i} m_{(i)} + r_{(i)}$ and $x' = m' + r' = \sum_{i=\sigma}^{\ell-1} 2^{\ell-1-i} m_{(i)} + r_{(i)}$, we have $x = x' + \delta$. If $x = x'$, then $\sigma = 0$. We use proof by contradiction to show that if $b = 1$, and $m_{(i)} \wedge r_{(i)} = 0$ for $i < \sigma$ and $m_{(\sigma)} \wedge r_{(\sigma)} = 1$, then $\delta = 0$. If $\delta \neq 0$, then there exists $\zeta < \sigma$ such that $\delta_{(\zeta)} \neq 0$, meaning $m_{(\zeta)} \oplus r_{(\zeta)} \oplus c_{(\zeta)} = 1$. Since $m_{(\zeta)} \wedge r_{(\zeta)} = 0$, at most one of $m_{(\zeta)}$ or $r_{(\zeta)}$ is 1. Thus, $m_{(\zeta)} + r_{(\zeta)} + c_{(\zeta)} = 1$, so $c_{(\zeta-1)} = 0$. Considering $m_{(\zeta-1)} \wedge r_{(\zeta-1)} = 0$, at most one of $m_{(\zeta-1)}$ or $r_{(\zeta-1)}$ is 1, so no carry is generated, and $c_{(\zeta-2)} = 0$. By recursion up to the highest bit, no carry is produced, so $b = 0$, which contradicts the condition. Then, we prove that the wrap-bit b' of x' is 0. Since $m'_{(0)} = 0$ and $r'_{(0)} = 0$, regardless of the lower-bit carry, no further carry can be generated, so $b' = 0$.

Next, we prove that in the case where $b = 0$, there exists an index σ such that $m_{(\sigma)} \vee r_{(\sigma)} = 1$ and $m_{(i)} \vee r_{(i)} = 0$ for $i < \sigma$. Then ζ can only be either σ or $\sigma - 1$.

First, we prove that there must exist a δ such that $m_{(\delta)} \vee r_{(\delta)} = 1$ and $m_{(i)} \vee r_{(i)} = 0$ for $i < \delta$. If such a δ does not exist, then for all i , $m_{(i)} \vee r_{(i)} = 0$, meaning $m_{(i)} = 0$ and $r_{(i)} = 0$, so $x = 0$, which contradicts the fact that x is non-zero. Therefore, δ exists.

We prove that for $a > 0$, if $\zeta = \delta + a$ or $\zeta = \delta - 1 - a$, then ζ cannot satisfy $x_{(\zeta)} = 1 \wedge \forall i < \zeta : x_{(i)} = 0$. If $\zeta = \delta + a$, since $m_{(\delta)} \vee r_{(\delta)} = 1$ and $b = 0$, x is not entirely zero from bit 0 to σ , which contradicts $m_{(i)} \vee r_{(i)} = 0$ for $i < \sigma$. If $\zeta = \delta - 1 - a$, since $m_{(i)} \vee r_{(i)} = 0$ for $i < \delta$, $m_{(\delta-1)} \vee r_{(\delta-1)} = 0$, no carry is generated at bit $\delta - 1 - a$, so $x_{(\delta-1-a)} = 0$, which contradicts the condition. $\square$

B.3 Security of First Non-zero Element Detection Protocol Π_{detect}

We define a standard functionality $\mathcal{F}_{\text{detect}}[p, K, \ell]$ of the first non-zero element detection as follows.

- $\mathcal{F}_{\text{detect}}[p, K, \ell]$ receives $[l_0], \dots, [l_{\ell-1}]$ from P_1 and P_2 , and reconstructs $l_0, \dots, l_{\ell-1}$ over $\mathbb{Z}_p$;
- $\mathcal{F}_{\text{detect}}[p, K, \ell]$ detects ζ such that $l_\zeta = 0$ for minimize ζ ;
- $\mathcal{F}_{\text{detect}}[p, K, \ell]$ generates $[\cdot]^K$ -share of ζ , and distributes $[\zeta]$ to P_1 and P_2

We the following theorem.

Theorem 3. *The protocol $\Pi_{\text{detect}}^{p,K,\ell}$ as depicted in Fig. 5 securely computes $\mathcal{F}_{\text{detect}}[p, K, \ell]$ against semi-honest PPT adversaries who can statically corrupt up to one party.*

Proof. For corrupted P_0 , $\mathcal{S}$ picks $\mathcal{L} = (s_0, \dots, s_{\ell-1}) \in (\mathbb{Z}_p^*)^\ell$, and random picks $\delta \in \mathbb{Z}_\ell$. $\mathcal{S}$ sets $s_\delta = 0$. Then $\mathcal{S}$ generates the $[\cdot]^M$ -share of $\mathcal{L}$ and plays the role of P_1 and P_2 to send $[\mathcal{L}]^p$ to P_0 .

We can easily demonstrate that the messages sent to P_0 in both the ideal and real worlds are indistinguishable. On one hand, in the protocol $\Pi_{\text{detect}}^{p,K,\ell}$, s is masked using $w \in \mathbb{Z}_p^*$, meaning that the numbers after the masking are uniformly random, except for 0. Moreover, there is exactly one 0 in s , and its position is randomized through the π permutation. Therefore, this is consistent with the list distribution sent by $\mathcal{S}$.

For corrupted P_1 (or P_2), we construct the simulation $\mathcal{S}$ as following (see Fig. 21). Since $([s_0]_1, \dots, [s_{\ell-1}]_1)$ is randomly picked by P_0 , the ideal world and real world are indistinguishable.

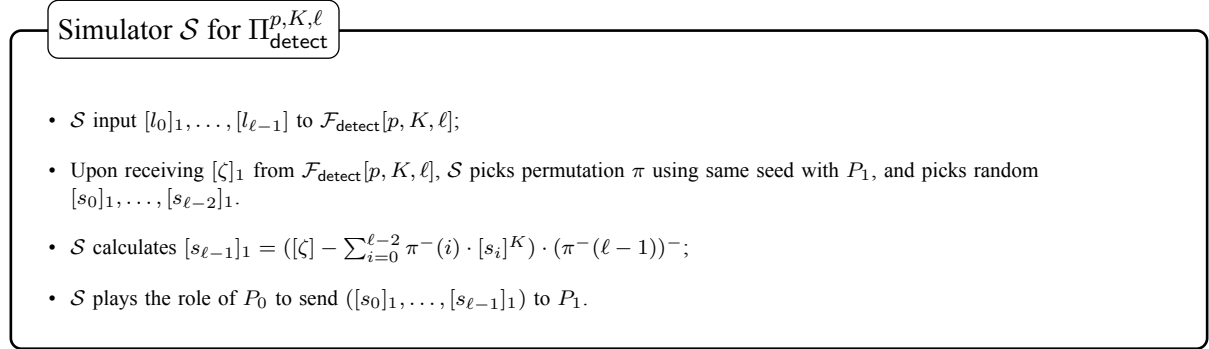


Figure 21: Simulator for $\Pi_{\text{detect}}^{p,K,\ell}$.

□

B.4 Security of Prefix List Computation Protocol Π_{prefix}

We define a standard functionality $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ of the prefix list computation as following.

- $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ receives $([l_0], \dots, [l_{\ell-1}])$ from P_1 and P_2 , and reconstructs $l_0, \dots, l_{\ell-1}$ over $\mathbb{Z}_p$; $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ receives $\langle b \rangle^K$ from all parties, and reconstructs b over $\mathbb{Z}_K$
- $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ detects ζ such that $l_\zeta = 0$ for minimize ζ ;
- $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ generates list $\mathcal{D} = (d_0, \dots, d_{\ell-1})$ while $d_i = b$ for $i \leq \zeta$ and $d_i = 0$ otherwise.
- $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ generates $[\cdot]^K$ -share of $\mathcal{D}$, and distributes $[\mathcal{D}]$ to P_1 and P_2

We have the following theorem.

Theorem 4. *The protocol $\Pi_{\text{prefix}}^{p,K,\ell}$ as depicted in Fig. 6 securely computes $\mathcal{F}_{\text{prefix}}[p, K, \ell]$ against semi-honest PPT adversaries who can statically corrupt up to one party.*

Simulator $\mathcal{S}$ for $\Pi_{\text{prefix}}^{p,K,\ell}$

- $\mathcal{S}$ input $[l_0]_1, \dots, [l_{\ell-1}]_1$ and $(m_b, [r_b]_1)$ to $\mathcal{F}_{\text{detect}}[p, K, \ell]$;
- Upon receiving $[d_0]_1, \dots, [d_{\ell-1}]_1$ from $\mathcal{F}_{\text{prefix}}[p, K, \ell]$, $\mathcal{S}$ picks permutation π using same seed with P_1 , and picks random $[s_0]_1, \dots, [s_{\ell-1}]_1$.
- $\mathcal{S}$ solves the system of equations $[d_j]^K = \sum_{i=0}^{\ell-1} d_j^{(i)} \cdot (m_b \cdot [s_i]^K + [\hat{s}_i]^K)$ to obtain $[\hat{s}_0]_1, \dots, [\hat{s}_{\ell-1}]_1$;
- $\mathcal{S}$ plays the role of P_0 to send $([s_0]_1, \dots, [s_{\ell-1}]_1)$ and $[\hat{s}_0]_1, \dots, [\hat{s}_{\ell-1}]_1$ to P_1 .

Figure 22: Simulator for $\Pi_{\text{prefix}}^{p,K,\ell}$.

Proof. The proof for the corrupted P_0 is exactly the same as Theorem 3. For corrupted P_1 (or P_2), we construct the simulation $\mathcal{S}$ as following (see Fig. 22). Since $([s_0]_1, \dots, [s_{\ell-1}]_1)$ and $[\hat{s}_0]_1, \dots, [\hat{s}_{\ell-1}]_1$ are randomly picked by P_0 , the ideal world and real world are indistinguishable. $\square$

B.5 Security of Power-of-Two Protocol Π_{pow2}

We define a standard functionality $\mathcal{F}_{\text{pow2}}[\ell]$ of the power-of-two computation as following.

- $\mathcal{F}_{\text{pow2}}[\ell]$ receives $[\zeta]^{2\ell}$ from P_1 and P_2 , and reconstructs ζ over $\mathbb{Z}_{2^\ell}$;
- $\mathcal{F}_{\text{pow2}}[\ell]$ calculates $z = 2^\zeta$ and generates $\langle \cdot \rangle$ -share of z ;
- $\mathcal{F}_{\text{pow2}}[\ell]$ distributes $\langle z \rangle$ to all parties.

To simplify the proof, we use $\mathcal{F}_{\text{mult}}$ to replace the multiplication protocol Π_{mult} in Π_{pow2}^ℓ . We the following theorem.

Theorem 5. *The protocol Π_{pow2}^ℓ as depicted in Fig. 7 securely computes $\mathcal{F}_{\text{pow2}}[\ell]$ in the $(\mathcal{F}_{\text{share}}, \mathcal{F}_{\text{mult}})$ -hybrid model against semi-honest PPT adversaries who can statically corrupt up to one party.*

Proof. For the corrupted P_0 , $\mathcal{S}$ receives $[r_z]_1$ and $[r_z]_2$ from $\mathcal{F}_{\text{pow2}}[\ell]$ and forwards them to P_0 when P_0 invoke $\mathcal{F}_{\text{share}}$. For corrupted P_1 (or P_2), we construct the simulation $\mathcal{S}$ as following (see Fig. 23).

Simulator $\mathcal{S}$ for Π_{pow2}^ℓ

- $\mathcal{S}$ input $[\zeta]_1$ to $\mathcal{F}_{\text{pow2}}[\ell]$;
- Upon receiving $(m_z, [r_z]_1)$ from $\mathcal{F}_{\text{pow2}}[\ell]$, $\mathcal{S}$ records it;
- Upon P_1 invoking $\mathcal{F}_{\text{share}}[2^\ell]$, $\mathcal{S}$ forwards $[r_z]_1$ to P_1 ;
- Upon P_1 invoking $\mathcal{F}_{\text{mult}}$, $\mathcal{S}$ sends random value to P_1 ;
- $\mathcal{S}$ calculates $\delta = m_z - ([2^\zeta]_1 - [r_z]_1)$ and plays the role of P_2 to send it to P_1 .

Figure 23: Simulator for Π_{pow2}^ℓ .

$\square$

C Related Work

For fixed-point arithmetic, most existing works [36, 34, 25, 43, 9, 14, 31, 53] follow the multiplication and truncation paradigm. To improve efficiency, many of these works employ probabilistic truncation, which introduces the need for padding with leading zeros. Recently, Dong et al. [16] proposed an efficient truncation scheme in the two-party setting, reducing the ring size required for truncation to a more compact form, though it requires the use of oblivious transfer. Liu et al. [28] studied a similar problem under Shamir secret sharing, but their scheme is limited by the need for secret sharing over Mersenne primes.

For the secure floating-point arithmetic, Aliasgari *et al.* [4] were the first to study secure floating-point evaluation. They implemented floating-point arithmetic in a three-party setting, but their scheme is a non-constant-round protocol, using techniques like bit extraction, leading to more than 34 rounds in the case of FP32. The same techniques were later applied in two-party computation scenarios [2], and in 2017, they extended this to the malicious threat model [3]. In two-party settings, some works [37, 19] explored using hybrid protocols for floating-point evaluation. Specifically, they implemented linear computations in floating-point using secret sharing and non-linear computations, such as left-shift operations, using garbled circuits. Building on this, Sasaki *et al.* [42] used the GMW protocol with Boolean secret sharing to implement non-linear computations. Rathee *et al.* [38] designed and implemented a floating-point library in a two-party setting, supporting a variety of mathematical operations, including basic floating-point operations such as addition, multiplication, division, and comparison, as well as more complex functions like trigonometric functions, exponentiation, and logarithms. Their work improved both the efficiency and precision compared to the previous two-party computation schemes. In subsequent work [39], they applied it to model training.